

Exam : **Associate-Developer-Apache-Spark-3.5**

Title : Databricks Certified
Associate Developer for
Apache Spark 3.5 - Python

Vendor : Databricks

Version : V12.65

QUESTION NO: 1

A Spark application developer wants to identify which operations cause shuffling, leading to a new stage in the Spark execution plan.

Which operation results in a shuffle and a new stage?

- A. `DataFrame.groupBy().agg()`
- B. `DataFrame.filter()`
- C. `DataFrame.withColumn()`
- D. `DataFrame.select()`

Answer: A

Explanation:

Operations that trigger data movement across partitions (like `groupBy`, `join`, `repartition`) result in a shuffle and a new stage.

From Spark documentation:

"groupBy and aggregation cause data to be shuffled across partitions to combine rows with the same key." Option A (`groupBy` + `agg`) → causes shuffle.

Options B, C, and D (`filter`, `withColumn`, `select`) → transformations that do not require shuffling; they are narrow dependencies.

Final answer: A

QUESTION NO: 2

12 of 55.

A data scientist has been investigating user profile data to build features for their model. After some exploratory data analysis, the data scientist identified that some records in the user profiles contain NULL values in too many fields to be useful.

The schema of the user profile table looks like this:

`user_id` STRING,
`username` STRING,
`date_of_birth` DATE,
`country` STRING,
`created_at` TIMESTAMP

The data scientist decided that if any record contains a NULL value in any field, they want to remove that record from the output before further processing.

Which block of Spark code can be used to achieve these requirements?

- A. `filtered_users = raw_users.na.drop("any")`
- B. `filtered_users = raw_users.na.drop("all")`
- C. `filtered_users = raw_users.dropna(how="any")`
- D. `filtered_users = raw_users.dropna(how="all")`

Answer: C

Explanation:

In Spark's `DataFrame` API, the `dropna()` (or equivalently, `DataFrameNaFunctions.drop()`) method removes rows containing null values.

Behavior:

`how="any"` → drops rows where any column has a null value.

`how="all"` → drops rows where all columns are null.

Since the data scientist wants to drop records with any null field, the correct parameter is `how="any"`.

Correct syntax:

```
filtered_users = raw_users.dropna(how="any")
```

This will remove all records that have at least one null value in any column.

Why the other options are incorrect:

A: Uses `na.drop("any")` but missing parentheses context (works only as `raw_users.na.drop("any")`, which is equivalent to option C).

B/D: `how="all"` only removes rows where all values are null - too strict for this use case.

Reference:

PySpark DataFrame API - `DataFrameNaFunctions.drop()` and `DataFrame.dropna()`.

Databricks Exam Guide (June 2025): Section "Developing Apache Spark DataFrame/DataSet API Applications" - covers handling missing data and DataFrame cleaning operations.

QUESTION NO: 3

An engineer has two DataFrames: `df1` (small) and `df2` (large). A broadcast join is used:

python

CopyEdit

```
from pyspark.sql.functions import broadcast
result = df2.join(broadcast(df1), on='id', how='inner')
```

What is the purpose of using `broadcast()` in this scenario?

Options:

- A. It filters the id values before performing the join.
- B. It increases the partition size for `df1` and `df2`.
- C. It reduces the number of shuffle operations by replicating the smaller DataFrame to all nodes.
- D. It ensures that the join happens only when the id values are identical.

Answer: C

Explanation:

`broadcast(df1)` tells Spark to send the small DataFrame (`df1`) to all worker nodes.

This eliminates the need for shuffling `df1` during the join.

Broadcast joins are optimized for scenarios with one large and one small table.

QUESTION NO: 4

A developer runs:

```
df.write.partitionBy("color", "fruit").parquet("/path/to/output")
```

What is the result?

Options:

- A. It stores all data in a single Parquet file.
- B. It throws an error if there are null values in either partition column.
- C. It appends new partitions to an existing Parquet file.
- D. It creates separate directories for each unique combination of color and fruit.

Answer: D

Explanation:

The `partitionBy()` method in Spark organizes output into subdirectories based on unique combinations of the specified columns:

e.g.

`/path/to/output/color=red/fruit=apple/part-0000.parquet`

`/path/to/output/color=green/fruit=banana/part-0001.parquet`

This improves query performance via partition pruning.

It does not consolidate into a single file.

Null values are allowed in partitions.

It does not "append" unless `.mode("append")` is used.

QUESTION NO: 5

A data scientist is analyzing a large dataset and has written a PySpark script that includes several transformations and actions on a `DataFrame`. The script ends with a `collect()` action to retrieve the results.

How does Apache Spark™'s execution hierarchy process the operations when the data scientist runs this script?

- A.** The script is first divided into multiple applications, then each application is split into jobs, stages, and finally tasks.
- B.** The entire script is treated as a single job, which is then divided into multiple stages, and each stage is further divided into tasks based on data partitions.
- C.** The `collect()` action triggers a job, which is divided into stages at shuffle boundaries, and each stage is split into tasks that operate on individual data partitions.
- D.** Spark creates a single task for each transformation and action in the script, and these tasks are grouped into stages and jobs based on their dependencies.

Answer: C

Explanation:

In Apache Spark, the execution hierarchy is structured as follows:

Application: The highest-level unit, representing the user program built on Spark.

Job: Triggered by an action (e.g., `collect()`, `count()`). Each action corresponds to a job.

Stage: A job is divided into stages based on shuffle boundaries. Each stage contains tasks that can be executed in parallel.

Task: The smallest unit of work, representing a single operation applied to a partition of the data.

When the `collect()` action is invoked, Spark initiates a job. This job is then divided into stages at points where data shuffling is required (i.e., wide transformations). Each stage comprises tasks that are distributed across the cluster's executors, operating on individual data partitions.

This hierarchical execution model allows Spark to efficiently process large-scale data by parallelizing tasks and optimizing resource utilization.

QUESTION NO: 6

47 of 55.

A data engineer has written the following code to join two `DataFrames` `df1` and `df2`:

```
df1 = spark.read.csv("sales_data.csv")
```

```
df2 = spark.read.csv("product_data.csv")
```

```
df_joined = df1.join(df2, df1.product_id == df2.product_id)
```

The DataFrame df1 contains ~10 GB of sales data, and df2 contains ~8 MB of product data. Which join strategy will Spark use?

- A. Shuffle join, as the size difference between df1 and df2 is too large for a broadcast join to work efficiently.
- B. Shuffle join, because AQE is not enabled, and Spark uses a static query plan.
- C. Shuffle join because no broadcast hints were provided.
- D. Broadcast join, as df2 is smaller than the default broadcast threshold.

Answer: D

Explanation:

Spark automatically uses a broadcast hash join when one side of the join is small enough to fit within the broadcast threshold.

Default threshold:

```
spark.sql.autoBroadcastJoinThreshold = 10MB (as of Spark 3.5)
```

Since df2 is 8 MB, Spark automatically broadcasts it to all executors. This avoids a shuffle on the large dataset (df1) and speeds up the join.

Why the other options are incorrect:

- A: 8 MB < 10 MB threshold → broadcast join is efficient.
- B: AQE is not required for broadcast joins; it's a static optimization.
- C: Broadcast hints are optional - Spark infers automatically.

Reference:

Databricks Exam Guide (June 2025): Section "Troubleshooting and Tuning Apache Spark DataFrame API Applications" - broadcast joins and optimization.

Spark SQL Join Strategies - Broadcast hash join and shuffle join thresholds.

QUESTION NO: 7

16 of 55.

A data engineer is reviewing a Spark application that applies several transformations to a DataFrame but notices that the job does not start executing immediately.

Which two characteristics of Apache Spark's execution model explain this behavior? (Choose 2 answers)

- A. Transformations are executed immediately to build the lineage graph.
- B. The Spark engine optimizes the execution plan during the transformations, causing delays.
- C. Transformations are evaluated lazily.
- D. The Spark engine requires manual intervention to start executing transformations.
- E. Only actions trigger the execution of the transformation pipeline.

Answer: C,E

Explanation:

Apache Spark follows a lazy evaluation model, meaning transformations (like filter(), select(), map()) are not executed immediately. Instead, they build a logical plan (lineage graph) that represents the sequence of operations to be applied.

Execution only begins when an action (e.g., count(), collect(), save(), show()) is called. At that

point, Spark's engine:

Optimizes the logical plan into a physical plan.

Divides it into stages and tasks.

Executes them across the cluster.

This design helps Spark optimize execution paths and avoid unnecessary computations.

Why the other options are incorrect:

A: Transformations do not execute immediately; they are deferred.

B: Optimization happens during job execution (after an action), not during transformations.

D: Execution starts automatically once an action is triggered, no manual intervention needed.

Reference:

Databricks Exam Guide (June 2025): Section "Apache Spark Architecture and Components" - covers lazy evaluation, actions vs. transformations, and execution hierarchy.

Spark 3.5 Documentation - Lazy Evaluation model and DAG scheduling.

QUESTION NO: 8

21 of 55.

What is the behavior of the function `date_sub(start, days)` if a negative value is passed into the `days` parameter?

A. The number of days specified will be added to the start date.

B. An error message of an invalid parameter will be returned.

C. The same start date will be returned.

D. The number of days specified will be removed from the start date.

Answer: A

Explanation:

In Spark SQL, the function `date_sub(startDate, days)` returns the date that is days before `startDate`.

If the `days` parameter is negative, Spark interprets it as subtracting a negative number, which effectively adds days to the date.

Example:

```
from pyspark.sql.functions import date_sub, lit
df = spark.createDataFrame([( "2024-10-01", )], ["dt"])
df.select(date_sub("dt", -5).alias("new_date")).show()
```

Output:

```
+-----+
| new_date |
+-----+
|2024-10-06|
+-----+
```

Why the other options are incorrect:

B: No error occurs; negative values are supported.

C: The start date changes if `days` $\neq$ 0.

D: Subtracting days would move the date backward, not forward.

Reference:

Spark SQL Functions - `date_sub(startDate, days)` and `date_add(startDate, days)` behavior.

Databricks Exam Guide (June 2025): Section "Using Spark SQL" - working with date and

timestamp functions.

QUESTION NO: 9

8 of 55.

A data scientist at a large e-commerce company needs to process and analyze 2 TB of daily customer transaction data. The company wants to implement real-time fraud detection and personalized product recommendations.

Currently, the company uses a traditional relational database system, which struggles with the increasing data volume and velocity.

Which feature of Apache Spark effectively addresses this challenge?

- A.** Ability to process small datasets efficiently
- B.** In-memory computation and parallel processing capabilities
- C.** Support for SQL queries on structured data
- D.** Built-in machine learning libraries

Answer: B

Explanation:

Apache Spark was designed for big data and high-velocity workloads. Its core strength lies in its in-memory computation and parallel distributed processing model.

These features allow Spark to:

Process large-scale datasets quickly across many nodes.

Support real-time and near-real-time analytics for tasks like fraud detection and recommendations.

Minimize disk I/O through caching and memory persistence.

Thus, the key advantage in this use case is Spark's ability to handle large data volumes efficiently using distributed, in-memory computation.

Why the other options are incorrect:

A: Spark is optimized for large, not small, datasets.

C: SQL support is useful but doesn't solve the scalability issue.

D: MLlib supports machine learning but relies on Spark's parallel computation for speed.

Reference:

Databricks Exam Guide (June 2025): Section "Apache Spark Architecture and Components" - identifies Spark's advantages: in-memory processing, distributed computation, and scalability.

Apache Spark 3.5 Overview - Key design goals and cluster computation model.

QUESTION NO: 10

29 of 55.

A Spark application is experiencing performance issues in client mode due to the driver being resource-constrained.

How should this issue be resolved?

- A.** Switch the deployment mode to cluster mode.
- B.** Add more executor instances to the cluster.
- C.** Increase the driver memory on the client machine.
- D.** Switch the deployment mode to local mode.

Answer: A

Explanation:

In client mode, the driver runs on the same machine that submitted the job (often a developer's workstation). If the driver has insufficient memory or CPU, it becomes a bottleneck.

Solution: Run the job in cluster mode.

In cluster mode, the driver runs inside the cluster on a worker node, benefiting from distributed cluster resources and improved performance for large workloads.

Why the other options are incorrect:

B: Executors handle tasks, not driver overhead.

C: May help temporarily but doesn't scale; cluster mode is best practice.

D: Local mode runs everything on one JVM - worse for large workloads.

Reference:

Databricks Exam Guide (June 2025): Section "Using Spark Connect to Deploy Applications" - explains client vs. cluster deployment modes.

Spark Deployment Overview - driver behavior and resource management.

QUESTION NO: 11

A Spark application suffers from too many small tasks due to excessive partitioning. How can this be fixed without a full shuffle?

Options:

A. Use the `distinct()` transformation to combine similar partitions

B. Use the `coalesce()` transformation with a lower number of partitions

C. Use the `sortBy()` transformation to reorganize the data

D. Use the `repartition()` transformation with a lower number of partitions

Answer: B

Explanation:

`coalesce(n)` reduces the number of partitions without triggering a full shuffle, unlike `repartition()`.

This is ideal when reducing partition count, especially during write operations.

QUESTION NO: 12

A developer needs to produce a Python dictionary using data stored in a small Parquet table, which looks like this:

region_id	region
1	AMERICA
3	EUROPE
0	AFRICA
4	MIDDLE EAST
2	ASIA

The resulting Python dictionary must contain a mapping of region -> region id containing the smallest 3 region_id values.

Which code fragment meets the requirements?

A)

```
regions = dict(  
    regions_df \  
        .select('region', 'region_id') \  
        .sort('region_id') \  
        .take(3)  
)
```

B)

```
)  
regions = dict(  
    regions_df \  
        .select('region_id', 'region') \  
        .sort('region_id') \  
        .take(3)  
)
```

C)

```
regions = dict(  
    regions_df \  
        .select('region_id', 'region') \  
        .limit(3) \  
        .collect()  
)
```

D)

```
regions = dict(  
    regions_df \  
        .select('region', 'region_id') \  
        .sort(desc('region_id')) \  
        .take(3)  
)
```

The resulting Python dictionary must contain a mapping of region -> region_id for the smallest 3 region_id values.

Which code fragment meets the requirements?

A.

```
regions = dict(  
    regions_df  
        .select('region', 'region_id')  
        .sort('region_id')  
        .take(3)  
)
```

B.

```
regions = dict(  
    regions_df  
        .select('region_id', 'region')  
        .sort('region_id')  
        .take(3)  
)
```

C.

```
regions = dict(  
    regions_df  
        .select('region_id', 'region')  
        .limit(3)
```

```
.collect()
)
```

D.

```
regions = dict(
regions_df
.select('region', 'region_id')
.sort(desc('region_id'))
.take(3)
)
```

Answer: A

Explanation:

The question requires creating a dictionary where keys are region values and values are the corresponding region_id integers. Furthermore, it asks to retrieve only the smallest 3 region_id values.

Key observations:

.select('region', 'region_id') puts the column order as expected by dict() - where the first column becomes the key and the second the value.

.sort('region_id') ensures sorting in ascending order so the smallest IDs are first.

.take(3) retrieves exactly 3 rows.

Wrapping the result in dict(...) correctly builds the required Python dictionary: { 'AFRICA': 0, 'AMERICA': 1, 'ASIA': 2 }.

Incorrect options:

Option B flips the order to region_id first, resulting in a dictionary with integer keys - not what's asked.

Option C uses .limit(3) without sorting, which leads to non-deterministic rows based on partition layout.

Option D sorts in descending order, giving the largest rather than smallest region_ids.

Hence, Option A meets all the requirements precisely.

QUESTION NO: 13

A data engineer is streaming data from Kafka and requires:

Minimal latency

Exactly-once processing guarantees

Which trigger mode should be used?

A. .trigger(processingTime='1 second')

B. .trigger(continuous=True)

C. .trigger(continuous='1 second')

D. .trigger(availableNow=True)

Answer: A

Explanation:

Exactly-once guarantees in Spark Structured Streaming require micro-batch mode (default), not continuous mode.

Continuous mode (.trigger(continuous=...)) only supports at-least-once semantics and lacks full fault-tolerance.

trigger(availableNow=True) is a batch-style trigger, not suited for low-latency streaming.

So:

Option A uses micro-batching with a tight trigger interval → minimal latency + exactly-once guarantee.

Final answer: A

QUESTION NO: 14

A developer notices that all the post-shuffle partitions in a dataset are smaller than the value set for `spark.sql.adaptive.maxShuffledHashJoinLocalMapThreshold`.

Which type of join will Adaptive Query Execution (AQE) choose in this case?

- A. A Cartesian join
- B. A shuffled hash join
- C. A broadcast nested loop join
- D. A sort-merge join

Answer: B

Explanation:

Adaptive Query Execution (AQE) dynamically selects join strategies based on actual data sizes at runtime. If the size of post-shuffle partitions is below the threshold set by:

`spark.sql.adaptive.maxShuffledHashJoinLocalMapThreshold`

then Spark prefers to use a shuffled hash join.

From the Spark documentation:

"AQE selects a shuffled hash join when the size of post-shuffle data is small enough to fit within the configured threshold, avoiding more expensive sort-merge joins." Therefore:

A is wrong - Cartesian joins are only used with no join condition.

B is correct - this is the optimized join for small partitioned shuffle data under AQE.

C and D are used under other scenarios but not for this case.

Final answer: B

QUESTION NO: 15

What is the behavior for function `date_sub(start, days)` if a negative value is passed into the `days` parameter?

- A. The same start date will be returned
- B. An error message of an invalid parameter will be returned
- C. The number of days specified will be added to the start date
- D. The number of days specified will be removed from the start date

Answer: C

Explanation:

The function `date_sub(start, days)` subtracts the number of days from the start date. If a negative number is passed, the behavior becomes a date addition.

Example:

```
SELECT date_sub('2024-05-01', -5)
```

```
-- Returns: 2024-05-06
```

So, a negative value effectively adds the absolute number of days to the date.

QUESTION NO: 16

14 of 55.

A developer created a DataFrame with columns color, fruit, and taste, and wrote the data to a Parquet directory using:

```
df.write.partitionBy("color", "taste").parquet("/path/to/output")
```

What is the result of this code?

- A. It appends new partitions to an existing Parquet file.
- B. It throws an error if there are null values in either partition column.
- C. It creates separate directories for each unique combination of color and taste.
- D. It stores all data in a single Parquet file.

Answer: C

Explanation:

When writing a DataFrame using `.partitionBy()` in Spark, the data is physically organized into directory structures corresponding to unique combinations of the partition columns.

Example:

```
/path/to/output/color=Red/taste=Sweet/part-0001.parquet
```

```
/path/to/output/color=Green/taste=Sour/part-0002.parquet
```

This structure improves query performance by pruning partitions when filtering on these columns.

Why the other options are incorrect:

- A: Appending requires `.mode("append")`, which isn't used here.
- B: Null values in partition columns are handled; they don't raise errors.
- D: Partitioning prevents storing all data in a single file.

Reference:

PySpark DataFrameWriter API - `partitionBy()` and `.parquet()` methods.

Databricks Exam Guide (June 2025): Section "Using Spark SQL" - partitioning and writing optimized output files.

QUESTION NO: 17

A developer initializes a SparkSession:

```
spark = SparkSession.builder \
    .appName("Analytics Application") \
    .getOrCreate()
```

```
spark = SparkSession.builder \
    .appName("Analytics Application") \
    .getOrCreate()
```

Which statement describes the spark SparkSession?

- A. The `getOrCreate()` method explicitly destroys any existing SparkSession and creates a new one.
- B. A SparkSession is unique for each appName, and calling `getOrCreate()` with the same name will return an existing SparkSession once it has been created.
- C. If a SparkSession already exists, this code will return the existing session instead of creating a new one.

D. A new SparkSession is created every time the getOrCreate() method is invoked.

Answer: C

Explanation:

According to the PySpark API documentation:

"getOrCreate(): Gets an existing SparkSession or, if there is no existing one, creates a new one based on the options set in this builder." This means Spark maintains a global singleton session within a JVM process. Repeated calls to getOrCreate() return the same session, unless explicitly stopped.

Option A is incorrect: the method does not destroy any session.

Option B incorrectly ties uniqueness to appName, which does not influence session reusability.

Option D is incorrect: it contradicts the fundamental behavior of getOrCreate().

(Source: PySpark SparkSession API Docs)

QUESTION NO: 18

In the code block below, aggDF contains aggregations on a streaming DataFrame:

```
1. aggDF \
2.     .writeStream \
3.     .outputMode("____") \
4.     .format("console") \
5.     .start()
```

Which output mode at line 3 ensures that the entire result table is written to the console during each trigger execution?

- A.** complete
- B.** append
- C.** replace
- D.** aggregate

Answer: A

Explanation:

The correct output mode for streaming aggregations that need to output the full updated results at each trigger is "complete".

From the official documentation:

"complete: The entire updated result table will be output to the sink every time there is a trigger." This is ideal for aggregations, such as counts or averages grouped by a key, where the result table changes incrementally over time.

append: only outputs newly added rows

replace and aggregate: invalid values for output mode

QUESTION NO: 19

26 of 55.

A data scientist at an e-commerce company is working with user data obtained from its

subscriber database and has stored the data in a DataFrame `df_user`.

Before further processing, the data scientist wants to create another DataFrame `df_user_non_pii` and store only the non-PII columns.

The PII columns in `df_user` are `name`, `email`, and `birthdate`.

Which code snippet can be used to meet this requirement?

A.

```
df_user_non_pii = df_user.drop("name", "email", "birthdate")
```

B.

```
df_user_non_pii = df_user.dropFields("name", "email", "birthdate")
```

C.

```
df_user_non_pii = df_user.select("name", "email", "birthdate")
```

D.

```
df_user_non_pii = df_user.remove("name", "email", "birthdate")
```

Answer: A

Explanation:

To exclude sensitive (PII) columns from a DataFrame, the easiest method is to use the `.drop()` function with the list of column names to remove.

Correct syntax:

```
df_user_non_pii = df_user.drop("name", "email", "birthdate")
```

This creates a new DataFrame containing all remaining columns.

Why the other options are incorrect:

B: `.dropFields()` is not valid for standard DataFrames - it's used for struct fields only.

C: `.select()` would keep only PII columns, not remove them.

D: `.remove()` does not exist in Spark DataFrame API.

Reference:

PySpark DataFrame API - `drop()` method for removing multiple columns.

Databricks Exam Guide (June 2025): Section "Developing Apache Spark

DataFrame/DataSet API Applications" - data manipulation, selecting, and dropping columns.

QUESTION NO: 20

What is the risk associated with this operation when converting a large Pandas API on Spark DataFrame back to a Pandas DataFrame?

A. The conversion will automatically distribute the data across worker nodes

B. The operation will fail if the Pandas DataFrame exceeds 1000 rows

C. Data will be lost during conversion

D. The operation will load all data into the driver's memory, potentially causing memory overflow

Answer: D

Explanation:

When you convert a large `pyspark.pandas` (aka Pandas API on Spark) DataFrame to a local Pandas DataFrame using `.toPandas()`, Spark collects all partitions to the driver.

From the Spark documentation:

"Be careful when converting large datasets to Pandas. The entire dataset will be pulled into the driver's memory." Thus, for large datasets, this can cause memory overflow or out-of-

memory errors on the driver.

Final answer: D

QUESTION NO: 21

A data engineer is running a batch processing job on a Spark cluster with the following configuration:

10 worker nodes

16 CPU cores per worker node

64 GB RAM per node

The data engineer wants to allocate four executors per node, each executor using four cores.

What is the total number of CPU cores used by the application?

A. 160

B. 64

C. 80

D. 40

Answer: C

Explanation:

If each of the 10 nodes runs 4 executors, and each executor is assigned 4 CPU cores:

Executors per node = 4

Cores per executor = 4

Total executors = $4 * 10 = 40$

Total cores = $40 \text{ executors} * 4 \text{ cores} = 160 \text{ cores}$

However, Spark uses 1 core for overhead on each node when managing multiple executors.

Thus, the practical allocation is:

Total usable executors = $4 \text{ executors/node} \times 10 \text{ nodes} = 40$

Total cores = $4 \text{ cores} \times 40 \text{ executors} = 160$

Answer : A - but the question asks specifically about "CPU cores used by the application," assuming all However, if you are considering $4 \text{ executors/node} \times 4 \text{ cores} = 16 \text{ cores per node}$, across 10 nodes, that's 160.

Final Answer: A

QUESTION NO: 22

48 of 55.

A data engineer needs to join multiple DataFrames and has written the following code:

```
from pyspark.sql.functions import broadcast
```

```
data1 = [(1, "A"), (2, "B")]
```

```
data2 = [(1, "X"), (2, "Y")]
```

```
data3 = [(1, "M"), (2, "N")]
```

```
df1 = spark.createDataFrame(data1, ["id", "val1"])
```

```
df2 = spark.createDataFrame(data2, ["id", "val2"])
```

```
df3 = spark.createDataFrame(data3, ["id", "val3"])
```

```
df_joined = df1.join(broadcast(df2), "id", "inner") \
```

```
.join(broadcast(df3), "id", "inner")
```

What will be the output of this code?

A. The code will work correctly and perform two broadcast joins simultaneously to join df1

with df2, and then the result with df3.

- B.** The code will fail because only one broadcast join can be performed at a time.
- C.** The code will fail because the second join condition (`df2.id == df3.id`) is incorrect.
- D.** The code will result in an error because `broadcast()` must be called before the joins, not inline.

Answer: A

Explanation:

Spark supports multiple broadcast joins in a single query plan, as long as each broadcasted DataFrame is small enough to fit under the configured threshold.

Execution Plan:

Spark broadcasts df2 to all executors.

Joins df1 (big) with broadcasted df2.

Then broadcasts df3 and performs another join with the intermediate result.

The result is efficient and avoids shuffling large data.

Why the other options are incorrect:

B: Multiple broadcast joins are supported in Spark 3.x.

C: The join condition is correct since all use id as the key.

D: `broadcast()` can be used inline; it's valid syntax.

Reference:

PySpark SQL Functions - `broadcast()` usage.

Databricks Exam Guide (June 2025): Section "Developing Apache Spark DataFrame/DataSet API Applications" - multiple broadcast join optimization.

QUESTION NO: 23

What is the benefit of Adaptive Query Execution (AQE)?

- A.** It allows Spark to optimize the query plan before execution but does not adapt during runtime.
- B.** It enables the adjustment of the query plan during runtime, handling skewed data, optimizing join strategies, and improving overall query performance.
- C.** It optimizes query execution by parallelizing tasks and does not adjust strategies based on runtime metrics like data skew.
- D.** It automatically distributes tasks across nodes in the clusters and does not perform runtime adjustments to the query plan.

Answer: B

Explanation:

Adaptive Query Execution (AQE) is a powerful optimization framework introduced in Apache Spark 3.0 and enabled by default since Spark 3.2. It dynamically adjusts query execution plans based on runtime statistics, leading to significant performance improvements. The key benefits of AQE include:

Dynamic Join Strategy Selection: AQE can switch join strategies at runtime. For instance, it can convert a sort-merge join to a broadcast hash join if it detects that one side of the join is small enough to be broadcasted, thus optimizing the join operation.

Handling Skewed Data: AQE detects skewed partitions during join operations and splits them into smaller partitions. This approach balances the workload across tasks, preventing scenarios where certain tasks take significantly longer due to data skew.

Coalescing Post-Shuffle Partitions: AQE dynamically coalesces small shuffle partitions into larger ones based on the actual data size, reducing the overhead of managing numerous small tasks and improving overall query performance .

These runtime optimizations allow Spark to adapt to the actual data characteristics during query execution, leading to more efficient resource utilization and faster query processing times.

QUESTION NO: 24

A data engineer writes the following code to join two DataFrames df1 and df2:

```
df1 = spark.read.csv("sales_data.csv") # ~10 GB
df2 = spark.read.csv("product_data.csv") # ~8 MB
result = df1.join(df2, df1.product_id == df2.product_id)
df1 = spark.read.csv("sales_data.csv")
df2 = spark.read.csv("product_data.csv")
result = df1.join(df2, df1.product_id == df2.product_id)
```

Which join strategy will Spark use?

- A. Shuffle join, because AQE is not enabled, and Spark uses a static query plan
- B. Broadcast join, as df2 is smaller than the default broadcast threshold
- C. Shuffle join, as the size difference between df1 and df2 is too large for a broadcast join to work efficiently
- D. Shuffle join because no broadcast hints were provided

Answer: B

Explanation:

The default broadcast join threshold in Spark is:

`spark.sql.autoBroadcastJoinThreshold = 10MB`

Since df2 is only 8 MB (less than 10 MB), Spark will automatically apply a broadcast join without requiring explicit hints.

From the Spark documentation:

"If one side of the join is smaller than the broadcast threshold, Spark will automatically broadcast it to all executors." A is incorrect because Spark does support auto broadcast even with static plans.

B is correct: Spark will automatically broadcast df2.

C and D are incorrect because Spark's default logic handles this optimization.

Final answer: B

QUESTION NO: 25

17 of 55.

A data engineer has noticed that upgrading the Spark version in their applications from Spark 3.0 to Spark 3.5 has improved the runtime of some scheduled Spark applications.

Looking further, the data engineer realizes that Adaptive Query Execution (AQE) is now enabled.

Which operation should AQE be implementing to automatically improve the Spark application performance?

- A. Dynamically switching join strategies

- B. Collecting persistent table statistics and storing them in the metastore for future use
- C. Improving the performance of single-stage Spark jobs
- D. Optimizing the layout of Delta files on disk

Answer: A

Explanation:

Adaptive Query Execution (AQE) in Spark 3.x automatically optimizes query plans at runtime based on the actual data characteristics observed during job execution.

Key features of AQE include:

Dynamic switching of join strategies: Changes between sort-merge join and broadcast join based on actual shuffle sizes.

Coalescing shuffle partitions: Reduces small tasks and improves parallelism efficiency.

Handling skew joins: Dynamically splits large partitions to avoid data skew.

Thus, the most accurate answer describing AQE's function is "dynamically switching join strategies." Why the other options are incorrect:

B: Table statistics are collected manually or by the metastore, not by AQE.

C: AQE benefits multi-stage jobs involving shuffles, not single-stage jobs.

D: Delta file optimization is handled by Databricks utilities, not AQE.

Reference:

Databricks Exam Guide (June 2025): Section "Troubleshooting and Tuning Apache Spark DataFrame API Applications" - covers AQE and its benefits.

Spark 3.5 Release Notes - Adaptive Query Execution dynamic optimizations.

QUESTION NO: 26

A data engineer wants to create a Streaming DataFrame that reads from a Kafka topic called feed.

```
1. spark
2. .readStream
3. .format("kafka")
4. .option("kafka.bootstrap.servers", "host1:port1,host2:port2")
5. _____
6. .load()
```

Which code fragment should be inserted in line 5 to meet the requirement?

Code context:

```
spark \
.readStream \
.format("kafka") \
.option("kafka.bootstrap.servers", "host1:port1,host2:port2") \
.[LINE 5] \
.load()
```

Options:

- A. `.option("subscribe", "feed")`
- B. `.option("subscribe.topic", "feed")`
- C. `.option("kafka.topic", "feed")`

D. `.option("topic", "feed")`

Answer: A

Explanation:

To read from a specific Kafka topic using Structured Streaming, the correct syntax is:

python

CopyEdit

`.option("subscribe", "feed")`

This is explicitly defined in the Spark documentation:

"subscribe - The Kafka topic to subscribe to. Only one topic can be specified for this option."

(Source: Apache Spark Structured Streaming + Kafka Integration Guide)

"subscribe - The Kafka topic to subscribe to. Only one topic can be specified for this option."

(Source: Apache Spark Structured Streaming + Kafka Integration Guide) B . "subscribe.topic" is invalid.

C . "kafka.topic" is not a recognized option.

D . "topic" is not valid for Kafka source in Spark.

QUESTION NO: 27

A developer is trying to join two tables, `sales.purchases_fct` and `sales.customer_dim`, using the following code:

```
import pyspark.sql.functions as F

purch_df = spark.table('sales.purchases_fct')
cust_df = spark.table('sales.customer_dim').dropDuplicates(['cust_id'])

fact_df = purch_df.join(cust_df, F.col('customer_id') == F.col('cust_id'))
```

`fact_df = purch_df.join(cust_df, F.col('customer_id') == F.col('custid'))` The developer has discovered that customers in the `purchases_fct` table that do not exist in the `customer_dim` table are being dropped from the joined table.

Which change should be made to the code to stop these customer records from being dropped?

A. `fact_df = purch_df.join(cust_df, F.col('customer_id') == F.col('custid'), 'left')`

B. `fact_df = cust_df.join(purch_df, F.col('customer_id') == F.col('custid'))`

C. `fact_df = purch_df.join(cust_df, F.col('cust_id') == F.col('customer_id'))`

D. `fact_df = purch_df.join(cust_df, F.col('customer_id') == F.col('custid'), 'right_outer')`

Answer: A

Explanation:

In Spark, the default join type is an inner join, which returns only the rows with matching keys in both DataFrames. To retain all records from the left DataFrame (`purch_df`) and include matching records from the right DataFrame (`cust_df`), a left outer join should be used.

By specifying the join type as 'left', the modified code ensures that all records from `purch_df` are preserved, and matching records from `cust_df` are included. Records in `purch_df` without a corresponding match in `cust_df` will have null values for the columns from `cust_df`.

This approach is consistent with standard SQL join operations and is supported in PySpark's DataFrame API.

QUESTION NO: 28

A data analyst wants to add a column date derived from a timestamp column.

Options:

- A. `dates_df.withColumn("date", f.unix_timestamp("timestamp")).show()`
- B. `dates_df.withColumn("date", f.to_date("timestamp")).show()`
- C. `dates_df.withColumn("date", f.date_format("timestamp", "yyyy-MM-dd")).show()`
- D. `dates_df.withColumn("date", f.from_unixtime("timestamp")).show()`

Answer: B

Explanation:

`f.to_date()` converts a timestamp or string to a `DateType`.

Ideal for extracting the date component (year-month-day) from a full timestamp.

Example:

```
from pyspark.sql.functions import to_date
dates_df.withColumn("date", to_date("timestamp"))
```

QUESTION NO: 29

An engineer wants to join two DataFrames `df1` and `df2` on the respective `employee_id` and `emp_id` columns:

`df1: employee_id INT, name STRING`

`df2: emp_id INT, department STRING`

The engineer uses:

```
result = df1.join(df2, df1.employee_id == df2.emp_id, how='inner')
```

What is the behaviour of the code snippet?

- A. The code fails to execute because the column names `employee_id` and `emp_id` do not match automatically
- B. The code fails to execute because it must use `on='employee_id'` to specify the join column explicitly
- C. The code fails to execute because PySpark does not support joining DataFrames with a different structure
- D. The code works as expected because the join condition explicitly matches `employee_id` from `df1` with `emp_id` from `df2`

Answer: D

Explanation:

In PySpark, when performing a join between two DataFrames, the columns do not have to share the same name. You can explicitly provide a join condition by comparing specific columns from each DataFrame.

This syntax is correct and fully supported:

```
df1.join(df2, df1.employee_id == df2.emp_id, how='inner')
```

This will perform an inner join between `df1` and `df2` using the `employee_id` from `df1` and `emp_id` from `df2`.

QUESTION NO: 30

An engineer has a large ORC file located at `/file/test_data.orc` and wants to read only specific columns to reduce memory usage.

Which code fragment will select the columns, i.e., col1, col2, during the reading process?

- A. `spark.read.orc("/file/test_data.orc").filter("col1 = 'value' ").select("col2")`
- B. `spark.read.format("orc").select("col1", "col2").load("/file/test_data.orc")`
- C. `spark.read.orc("/file/test_data.orc").selected("col1", "col2")`
- D. `spark.read.format("orc").load("/file/test_data.orc").select("col1", "col2")`

Answer: D

Explanation:

The correct way to load specific columns from an ORC file is to first load the file using `.load()` and then apply `.select()` on the resulting DataFrame. This is valid with `.read.format("orc")` or the shortcut `.read.orc()`.

`df = spark.read.format("orc").load("/file/test_data.orc").select("col1", "col2")` Why others are incorrect:

A performs selection after filtering, but doesn't match the intention to minimize memory at load.

B incorrectly tries to use `.select()` before `.load()`, which is invalid.

C uses a non-existent `.selected()` method.

D correctly loads and then selects.

QUESTION NO: 31

32 of 55.

A developer is creating a Spark application that performs multiple DataFrame transformations and actions. The developer wants to maintain optimal performance by properly managing the SparkSession.

How should the developer handle the SparkSession throughout the application?

- A. Use a single SparkSession instance for the entire application.
- B. Avoid using a SparkSession and rely on SparkContext only.
- C. Create a new SparkSession instance before each transformation.
- D. Stop and restart the SparkSession after each action.

Answer: A

Explanation:

The SparkSession is the entry point to Spark functionality in modern versions (2.x and later). It unifies the SparkContext, SQLContext, and HiveContext into a single object.

Best Practice:

Use one SparkSession for the entire application.

Create it once at the start using `SparkSession.builder.getOrCreate()`.

Reuse it across all transformations and actions.

Stop it only after all operations are completed.

Example:

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("MyApp").getOrCreate()
# Perform transformations and actions
spark.stop()
```

Why the other options are incorrect:

B: SparkSession is the recommended interface; SparkContext alone is deprecated for

SQL/DataFrame APIs.

C: Creating multiple sessions increases overhead and wastes resources.

D: Restarting SparkSession breaks lineage and adds unnecessary startup costs.

Reference:

Spark API Reference - SparkSession lifecycle.

Databricks Exam Guide (June 2025): Section "Apache Spark Architecture and Components" - explains SparkSession lifecycle and application management.

QUESTION NO: 32

33 of 55.

The data engineering team created a pipeline that extracts data from a transaction system. The transaction system stores timestamps in UTC, and the data engineers must now transform the transaction_datetime field to the "America/New_York" timezone for reporting. Which code should be used to convert the timestamp to the target timezone?

A.

```
raw.withColumn("transaction_datetime", from_utc_timestamp(col("transaction_datetime"), "America/New_York"))
```

B.

```
raw.withColumn("transaction_datetime", to_utc_timestamp(col("transaction_datetime"), "America/New_York"))
```

C.

```
raw.withColumn("transaction_datetime", date_format(col("transaction_datetime"), "America/New_York"))
```

D.

```
raw.withColumn("transaction_datetime", convert_timezone(col("transaction_datetime"), "America/New_York"))
```

Answer: A

Explanation:

In Spark SQL, to convert a UTC timestamp to another timezone, you use the function from_utc_timestamp().

Correct syntax:

```
from pyspark.sql.functions import from_utc_timestamp, col
df_converted = raw.withColumn(
    "transaction_datetime",
    from_utc_timestamp(col("transaction_datetime"), "America/New_York")
)
```

This adjusts the UTC time into the specified timezone using Spark's timezone database.

Why the other options are incorrect:

B: to_utc_timestamp() converts local time to UTC, not the other way around.

C: date_format() formats timestamps as strings but doesn't adjust timezones.

D: convert_timezone() is not a valid Spark SQL function.

Reference:

Spark SQL Functions - from_utc_timestamp() and to_utc_timestamp().

Databricks Exam Guide (June 2025): Section "Using Spark SQL" - working with timestamps and timezone conversions.

QUESTION NO: 33

Given the code fragment:

```
import pyspark.pandas as ps
psdf = ps.DataFrame({'col1': [1, 2], 'col2': [3, 4]})
```

```
import pyspark.pandas as ps
```

```
psdf = ps.DataFrame({'col1': [1, 2], 'col2': [3, 4]})
```

Which method is used to convert a Pandas API on Spark DataFrame (pyspark.pandas.DataFrame) into a standard PySpark DataFrame (pyspark.sql.DataFrame)?

- A. psdf.to_spark()
- B. psdf.to_pyspark()
- C. psdf.to_pandas()
- D. psdf.to_dataframe()

Answer: A

Explanation:

Pandas API on Spark (pyspark.pandas) allows interoperability with PySpark DataFrames. To convert a pyspark.pandas.DataFrame to a standard PySpark DataFrame, you use `.to_spark()`.

Example:

```
df = psdf.to_spark()
```

This is the officially supported method as per Databricks Documentation.

Incorrect options:

B, D: Invalid or nonexistent methods.

C: Converts to a local pandas DataFrame, not a PySpark DataFrame.

QUESTION NO: 34

An engineer notices a significant increase in the job execution time during the execution of a Spark job. After some investigation, the engineer decides to check the logs produced by the Executors.

How should the engineer retrieve the Executor logs to diagnose performance issues in the Spark application?

- A. Locate the executor logs on the Spark master node, typically under the `/tmp` directory.
- B. Use the command `spark-submit` with the `-verbose` flag to print the logs to the console.
- C. Use the Spark UI to select the stage and view the executor logs directly from the stages tab.
- D. Fetch the logs by running a Spark job with the `spark-sql` CLI tool.

Answer: C

Explanation:

The Spark UI is the standard and most effective way to inspect executor logs, task time, input size, and shuffles.

From the Databricks documentation:

"You can monitor job execution via the Spark Web UI. It includes detailed logs and metrics, including task-level execution time, shuffle reads/writes, and executor memory usage."

(Source: Databricks Spark Monitoring Guide) Option A is incorrect: logs are not guaranteed to be in /tmp, especially in cloud environments.

B . -verbose helps during job submission but doesn't give detailed executor logs.

D . spark-sql is a CLI tool for running queries, not for inspecting logs.

Hence, the correct method is using the Spark UI → Stages tab → Executor logs.

QUESTION NO: 35

An application architect has been investigating Spark Connect as a way to modernize existing Spark applications running in their organization.

Which requirement blocks the adoption of Spark Connect in this organization?

A. Debuggability: the ability to perform interactive debugging directly from the application code

B. Upgradability: the ability to upgrade the Spark applications independently from the Spark driver itself

C. Complete Spark API support: the ability to migrate all existing code to Spark Connect without modification, including the RDD APIs

D. Stability: isolation of application code and dependencies from each other and the Spark driver

Answer: C

Explanation:

Spark Connect enables a decoupled client-server architecture, allowing remote clients to run Spark code via gRPC.

However, as of Spark 3.5, Spark Connect supports DataFrame and SQL APIs, but not RDD APIs.

Limitation:

Applications that rely heavily on RDD-based transformations or actions cannot be migrated directly to Spark Connect.

These APIs require tight driver integration, which Spark Connect intentionally decouples.

Thus, complete Spark API compatibility is not yet achieved - this is the key adoption blocker.

Why the other options are incorrect:

A: Debugging is possible through IDE integration and logs on the client side.

B: Spark Connect actually supports upgradable clients independent of the driver - this is an advantage, not a limitation.

D: Spark Connect provides strong isolation between the client and driver processes.

Reference:

Spark 3.5 Documentation - Spark Connect architecture and supported APIs.

Databricks Exam Guide (June 2025): Section "Using Spark Connect to Deploy Applications" - Spark Connect limitations (no RDD API support).

QUESTION NO: 36

A Spark engineer is troubleshooting a Spark application that has been encountering out-of-memory errors during execution. By reviewing the Spark driver logs, the engineer notices multiple "GC overhead limit exceeded" messages.

Which action should the engineer take to resolve this issue?

A. Optimize the data processing logic by repartitioning the DataFrame.

- B. Modify the Spark configuration to disable garbage collection
- C. Increase the memory allocated to the Spark Driver.
- D. Cache large DataFrames to persist them in memory.

Answer: C

Explanation:

The message "GC overhead limit exceeded" typically indicates that the JVM is spending too much time in garbage collection with little memory recovery. This suggests that the driver or executor is under-provisioned in memory.

The most effective remedy is to increase the driver memory using:

--driver-memory 4g

This is confirmed in Spark's official troubleshooting documentation:

"If you see a lot of GC overhead limit exceeded errors in the driver logs, it's a sign that the driver is running out of memory."

- Spark Tuning Guide

"If you see a lot of GC overhead limit exceeded errors in the driver logs, it's a sign that the driver is running out of memory."

- Spark Tuning Guide

Why others are incorrect:

A may help but does not directly address the driver memory shortage.

B is not a valid action; GC cannot be disabled.

D increases memory usage, worsening the problem.

QUESTION NO: 37

49 of 55.

In the code block below, aggDF contains aggregations on a streaming DataFrame:

```
aggDF.writeStream \  
  .format("console") \  
  .outputMode("???") \  
  .start()
```

Which output mode at line 3 ensures that the entire result table is written to the console during each trigger execution?

- A. AGGREGATE
- B. COMPLETE
- C. REPLACE
- D. APPEND

Answer: B

Explanation:

Structured Streaming supports three output modes:

Append: Writes only new rows since the last trigger.

Update: Writes only updated rows.

Complete: Writes the entire result table after every trigger execution.

For aggregations like `groupBy().count()`, only complete mode outputs the entire table each time.

Example:

```
aggDF.writeStream \  

```

```
.outputMode("complete") \  
.format("console") \  
.start()
```

Why the other options are incorrect:

A: "AGGREGATE" is not a valid output mode.

C: "REPLACE" does not exist.

D: "APPEND" writes only new rows, not the full table.

Reference:

PySpark Structured Streaming - Output Modes (append, update, complete).

Databricks Exam Guide (June 2025): Section "Structured Streaming" - output modes and use cases for aggregations.

QUESTION NO: 38

31 of 55.

Given a DataFrame df that has 10 partitions, after running the code:

```
df.repartition(20)
```

How many partitions will the result DataFrame have?

A. 5

B. 20

C. Same number as the cluster executors

D. 10

Answer: B

Explanation:

The repartition(n) transformation reshuffles data into exactly n partitions.

Unlike coalesce(), repartition() always causes a shuffle to evenly redistribute the data.

Correct behavior:

```
df2 = df.repartition(20)
```

```
df2.rdd.getNumPartitions() # returns 20
```

Thus, the resulting DataFrame will have 20 partitions.

Why the other options are incorrect:

A/D: Doesn't retain old partition count - it's explicitly set to 20.

C: Number of partitions is not automatically tied to executors.

Reference:

PySpark DataFrame API - repartition() vs. coalesce().

Databricks Exam Guide (June 2025): Section "Developing Apache Spark

DataFrame/DataSet API Applications" - tuning partitioning and shuffling for performance.

QUESTION NO: 39

A data engineer is working on a Streaming DataFrame streaming_df with the given streaming data:

Id	Name	count	timestamp
1	Delhi	20	2024-09-19T10:10:10.000+00:00
1	Delhi	50	2024-09-19T10:10:50.000+00:00
1	Delhi	10	2024-09-19T10:11:10.000+00:00
2	London	50	2024-09-19T10:10:20.000+00:00
3	Paris	30	2024-09-19T10:10:30.000+00:00
3	Paris	20	2024-09-19T10:11:20.000+00:00
4	Washington	10	2024-09-19T10:10:40.000+00:00
4	Washington	40	2024-09-19T10:11:00.000+00:00

Which operation is supported with streamingdf ?

- A. `streaming_df. select (countDistinct ("Name"))`
- B. `streaming_df.groupby("Id").count ()`
- C. `streaming_df.orderBy("timestamp").limit(4)`
- D. `streaming_df.filter (col("count") < 30).show()`

Answer: B

In Structured Streaming, only a limited subset of operations is supported due to the nature of unbounded data. Operations like sorting (`orderBy`) and global aggregation (`countDistinct`) require a full view of the dataset, which is not possible with streaming data unless specific watermarks or windows are defined.

Review of Each Option:

A: `select(countDistinct("Name"))`

Not allowed - Global aggregation like `countDistinct()` requires the full dataset and is not supported directly in streaming without watermark and windowing logic.

Reference: Databricks Structured Streaming Guide - Unsupported Operations.

B: `groupby("Id").count()`

Supported - Streaming aggregations over a key (like `groupBy("Id")`) are supported. Spark maintains intermediate state for each key.

Reference: Databricks Docs → Aggregations in Structured Streaming (<https://docs.databricks.com/structured-streaming/aggregation.html>) C .

`orderBy("timestamp").limit(4)`

Not allowed - Sorting and limiting require a full view of the stream (which is infinite), so this is unsupported in streaming DataFrames.

Reference: Spark Structured Streaming - Unsupported Operations (ordering without watermark/window not allowed).

D: `filter(col("count") < 30).show()`

Not allowed - `show()` is a blocking operation used for debugging batch DataFrames; it's not allowed on streaming DataFrames.

Reference: Structured Streaming Programming Guide - Output operations like show() are not supported.

Reference Extract from Official Guide:

"Operations like orderBy, limit, show, and countDistinct are not supported in Structured Streaming because they require the full dataset to compute a result. Use groupBy(...).agg(...) instead for incremental aggregations."

- Databricks Structured Streaming Programming Guide

QUESTION NO: 40

Given the code:

```
df = spark.read.csv("large_dataset.csv")
filtered_df = df.filter(col("error_column")
                        .contains("error"))
mapped_df = filtered_df.select(split(col("timestamp"), " ").getItem(0).alias("date"),
                               lit(1).alias("count"))
reduced_df = mapped_df.groupBy("date").sum("count")
reduced_df.count()
reduced_df.show()
```

df = spark.read.csv("large_dataset.csv")
 filtered_df = df.filter(col("error_column").contains("error"))
 mapped_df = filtered_df.select(split(col("timestamp"), " ").getItem(0).alias("date"),
 lit(1).alias("count")) reduced_df = mapped_df.groupBy("date").sum("count")
 reduced_df.count() reduced_df.show() At which point will Spark actually begin processing the data?

- A. When the filter transformation is applied
- B. When the count action is applied
- C. When the groupBy transformation is applied
- D. When the show action is applied

Answer: B

Explanation:

Spark uses lazy evaluation. Transformations like filter, select, and groupBy only define the DAG (Directed Acyclic Graph). No execution occurs until an action is triggered.

The first action in the code is: reduced_df.count()

So Spark starts processing data at this line.

QUESTION NO: 41

A Spark DataFrame df is cached using the MEMORY_AND_DISK storage level, but the DataFrame is too large to fit entirely in memory.

What is the likely behavior when Spark runs out of memory to store the DataFrame?

- A. Spark duplicates the DataFrame in both memory and disk. If it doesn't fit in memory, the DataFrame is stored and retrieved from the disk entirely.
- B. Spark splits the DataFrame evenly between memory and disk, ensuring balanced storage utilization.
- C. Spark will store as much data as possible in memory and spill the rest to disk when memory is full, continuing processing with performance overhead.
- D. Spark stores the frequently accessed rows in memory and less frequently accessed rows

on disk, utilizing both resources to offer balanced performance.

Answer: C

Explanation:

When using the MEMORY_AND_DISK storage level, Spark attempts to cache as much of the DataFrame in memory as possible. If the DataFrame does not fit entirely in memory, Spark will store the remaining partitions on disk. This allows processing to continue, albeit with a performance overhead due to disk I/O.

As per the Spark documentation:

"MEMORY_AND_DISK: It stores partitions that do not fit in memory on disk and keeps the rest in memory. This can be useful when working with datasets that are larger than the available memory."

- Perflcient Blogs: Spark - StorageLevel

This behavior ensures that Spark can handle datasets larger than the available memory by spilling excess data to disk, thus preventing job failures due to memory constraints.

QUESTION NO: 42

An MLOps engineer is building a Pandas UDF that applies a language model that translates English strings into Spanish. The initial code is loading the model on every call to the UDF, which is hurting the performance of the data pipeline.

The initial code is:

```
def in_spanish_inner(df: pd.Series) -> pd.Series:
    model = get_translation_model(target_lang='es')
    return df.apply(model)
```

```
in_spanish = sf.pandas_udf(in_spanish_inner, StringType())
```

```
def in_spanish_inner(df: pd.Series) -> pd.Series:
```

```
    model = get_translation_model(target_lang='es')
```

```
    return df.apply(model)
```

```
in_spanish = sf.pandas_udf(in_spanish_inner, StringType())
```

How can the MLOps engineer change this code to reduce how many times the language model is loaded?

- A. Convert the Pandas UDF to a PySpark UDF
- B. Convert the Pandas UDF from a Series → Series UDF to a Series → Scalar UDF
- C. Run the in_spanish_inner() function in a mapInPandas() function call
- D. Convert the Pandas UDF from a Series → Series UDF to an Iterator[Series] → Iterator[Series] UDF

Answer: D

Explanation:

The provided code defines a Pandas UDF of type Series-to-Series, where a new instance of the language model is created on each call, which happens per batch. This is inefficient and results in significant overhead due to repeated model initialization.

To reduce the frequency of model loading, the engineer should convert the UDF to an iterator-based Pandas UDF (Iterator[pd.Series] -> Iterator[pd.Series]). This allows the model

to be loaded once per executor and reused across multiple batches, rather than once per call.

From the official Databricks documentation:

"Iterator of Series to Iterator of Series UDFs are useful when the UDF initialization is expensive... For example, loading a ML model once per executor rather than once per row/batch."

- Databricks Official Docs: Pandas UDFs

Correct implementation looks like:

python

CopyEdit

```
@pandas_udf("string")
```

```
def translate_udf(batch_iter: Iterator[pd.Series]) -> Iterator[pd.Series]:
```

```
    model = get_translation_model(target_lang='es')
```

```
    for batch in batch_iter:
```

```
        yield batch.apply(model)
```

This refactor ensures the `get_translation_model()` is invoked once per executor process, not per batch, significantly improving pipeline performance.

QUESTION NO: 43

Given the following code snippet in `my_spark_app.py`:

```
from pyspark.sql
import SparkSession

spark = SparkSession.builder.appName("CoreComponentsExample").getOrCreate()

data = [("Alice", 34), ("Bob", 36), ("Cathy", 31)]
columns = ["Name", "Age"]

df = spark.createDataFrame(data, columns).withColumn("Status", "Pass")
df_filtered = df.filter(df.Age > 35)
df_filtered.show()

spark.stop()
```

What is the role of the driver node?

- A.** The driver node orchestrates the execution by transforming actions into tasks and distributing them to worker nodes
- B.** The driver node only provides the user interface for monitoring the application
- C.** The driver node holds the DataFrame data and performs all computations locally
- D.** The driver node stores the final result after computations are completed by worker nodes

Answer: A

Explanation:

In the Spark architecture, the driver node is responsible for orchestrating the execution of a Spark application. It converts user-defined transformations and actions into a logical plan, optimizes it into a physical plan, and then splits the plan into tasks that are distributed to the executor nodes.

As per Databricks and Spark documentation:

"The driver node is responsible for maintaining information about the Spark application, responding to a user's program or input, and analyzing, distributing, and scheduling work across the executors." This means:

Option A is correct because the driver schedules and coordinates the job execution.

Option B is incorrect because the driver does more than just UI monitoring.

Option C is incorrect since data and computations are distributed across executor nodes.

Option D is incorrect; results are returned to the driver but not stored long-term by it.

QUESTION NO: 44

5 of 55.

What is the relationship between jobs, stages, and tasks during execution in Apache Spark?

- A. A job contains multiple tasks, and each task contains multiple stages.
- B. A stage contains multiple jobs, and each job contains multiple tasks.
- C. A stage contains multiple tasks, and each task contains multiple jobs.
- D. A job contains multiple stages, and each stage contains multiple tasks.

Answer: D

Explanation:

In Apache Spark's execution hierarchy, the relationships are structured as follows:

Job: Created when an action (e.g., count(), collect(), save()) is triggered on an RDD or DataFrame.

Stage: Each job is divided into one or more stages, separated by shuffle boundaries (e.g., after a reduceByKey or join).

Task: Each stage consists of multiple tasks, one per partition, executed in parallel on executors.

Execution Hierarchy:

Job → Stage(s) → Task(s)

So, a job contains multiple stages, and each stage contains multiple tasks.

Why the other options are incorrect:

A: A job does not directly contain tasks without stages.

B: A stage cannot contain multiple jobs; it belongs to a single job.

C: Tasks do not contain jobs.

Reference (Databricks Apache Spark 3.5 - Python / Study Guide):

Spark Architecture Overview - Execution Hierarchy: Jobs, Stages, and Tasks.

Databricks Exam Guide (June 2025): Section "Apache Spark Architecture and Components" - describes execution hierarchy and lazy evaluation.

QUESTION NO: 45

A data engineer observes that an upstream streaming source sends duplicate records, where duplicates share the same key and have at most a 30-minute difference in event_timestamp.

The engineer adds:

```
dropDuplicatesWithinWatermark("event_timestamp", "30 minutes")
```

What is the result?

- A. It is not able to handle deduplication in this scenario
- B. It removes duplicates that arrive within the 30-minute window specified by the watermark

- C. It removes all duplicates regardless of when they arrive
- D. It accepts watermarks in seconds and the code results in an error

Answer: B

Explanation:

The method `dropDuplicatesWithinWatermark()` in Structured Streaming drops duplicate records based on a specified column and watermark window. The watermark defines the threshold for how late data is considered valid.

From the Spark documentation:

"`dropDuplicatesWithinWatermark` removes duplicates that occur within the event-time watermark window." In this case, Spark will retain the first occurrence and drop subsequent records within the 30-minute watermark window.

Final answer: B

QUESTION NO: 46

27 of 55.

A data engineer needs to add all the rows from one table to all the rows from another, but not all the columns in the first table exist in the second table.

The error message is:

`AnalysisException: UNION can only be performed on tables with the same number of columns.`

The existing code is:

```
au_df.union(nz_df)
```

The DataFrame `au_df` has one extra column that does not exist in the DataFrame `nz_df`, but otherwise both DataFrames have the same column names and data types.

What should the data engineer fix in the code to ensure the combined DataFrame can be produced as expected?

A.

```
df = au_df.unionByName(nz_df, allowMissingColumns=True)
```

B.

```
df = au_df.unionAll(nz_df)
```

C.

```
df = au_df.unionByName(nz_df, allowMissingColumns=False)
```

D.

```
df = au_df.union(nz_df, allowMissingColumns=True)
```

Answer: A

Explanation:

When two DataFrames have different column sets, the normal `union()` or `unionAll()` functions fail unless both have exactly the same columns in the same order.

Solution: Use `unionByName()` with `allowMissingColumns=True`.

This aligns columns by name and automatically adds missing columns with null values.

Correct syntax:

```
combined_df = au_df.unionByName(nz_df, allowMissingColumns=True)
```

This ensures the union works even if one DataFrame has extra or missing columns.

Why the other options are incorrect:

B: `unionAll()` is deprecated; also requires identical schemas.

C: With allowMissingColumns=False, Spark still throws a mismatch error.

D: union() doesn't accept the allowMissingColumns argument.

Reference:

PySpark API - DataFrame.unionByName() with allowMissingColumns option.

Databricks Exam Guide (June 2025): Section "Developing Apache Spark

DataFrame/DataSet API Applications" - combining DataFrames and schema alignment.

QUESTION NO: 47

A data engineer needs to write a Streaming DataFrame as Parquet files.

Given the code:

```
df
  .writeStream
  // -- insert code here --
  .checkpointLocation("path/to/checkpoint/dir")
  .start()
```

Which code fragment should be inserted to meet the requirement?

A)

```
.format("parquet")
.option("location", "path/to/destination/dir")
```

B)

```
.option("format", "parquet")
.option("destination", "path/to/destination/dir")
```

C)

```
.option("format", "parquet")
.option("location", "path/to/dest
```

D)

```
.format("parquet")
.option("path", "path/to/destinat
```

Which code fragment should be inserted to meet the requirement?

A.

```
.format("parquet")
.option("location", "path/to/destination/dir")
```

B.

CopyEdit

```
.option("format", "parquet")
.option("destination", "path/to/destination/dir")
```

C.

```
.option("format", "parquet")
.option("location", "path/to/destination/dir")
```

D.

```
.format("parquet")
.option("path", "path/to/destination/dir")
```

Answer: D

Explanation:

To write a structured streaming DataFrame to Parquet files, the correct way to specify the format and output directory is:

```
.writeStream
.format("parquet")
.option("path", "path/to/destination/dir")
```

According to Spark documentation:

"When writing to file-based sinks (like Parquet), you must specify the path using the `.option("path", ...)` method. Unlike batch writes, `.save()` is not supported." Option A incorrectly uses `.option("location", ...)` (invalid for Parquet sink).

Option B incorrectly sets the format via `.option("format", ...)`, which is not the correct method.

Option C repeats the same issue.

Option D is correct: `.format("parquet") + .option("path", ...)` is the required syntax.

Final answer: D

QUESTION NO: 48

A data engineer is building a Structured Streaming pipeline and wants the pipeline to recover from failures or intentional shutdowns by continuing where the pipeline left off.

How can this be achieved?

- A. By configuring the option `checkpointLocation` during `readStream`
- B. By configuring the option `recoveryLocation` during the `SparkSession` initialization
- C. By configuring the option `recoveryLocation` during `writeStream`
- D. By configuring the option `checkpointLocation` during `writeStream`

Answer: D

Explanation:

To enable a Structured Streaming query to recover from failures or intentional shutdowns, it is essential to specify the `checkpointLocation` option during the `writeStream` operation. This checkpoint location stores the progress information of the streaming query, allowing it to resume from where it left off.

According to the Databricks documentation:

"You must specify the `checkpointLocation` option before you run a streaming query, as in the following example:

```
.option("checkpointLocation", "/path/to/checkpoint/dir")
.toTable("catalog.schema.table")
```

- Databricks Documentation: Structured Streaming checkpoints

By setting the `checkpointLocation` during `writeStream`, Spark can maintain state information and ensure exactly-once processing semantics, which are crucial for reliable streaming applications.

QUESTION NO: 49

A Spark engineer must select an appropriate deployment mode for the Spark jobs.

What is the benefit of using cluster mode in Apache Spark™?

- A.** In cluster mode, resources are allocated from a resource manager on the cluster, enabling better performance and scalability for large jobs
- B.** In cluster mode, the driver is responsible for executing all tasks locally without distributing them across the worker nodes.
- C.** In cluster mode, the driver runs on the client machine, which can limit the application's ability to handle large datasets efficiently.
- D.** In cluster mode, the driver program runs on one of the worker nodes, allowing the application to fully utilize the distributed resources of the cluster.

Answer: D

Explanation:

In Apache Spark's cluster mode:

"The driver program runs on the cluster's worker node instead of the client's local machine. This allows the driver to be close to the data and other executors, reducing network overhead and improving fault tolerance for production jobs." (Source: Apache Spark documentation - Cluster Mode Overview)

"The driver program runs on the cluster's worker node instead of the client's local machine. This allows the driver to be close to the data and other executors, reducing network overhead and improving fault tolerance for production jobs." (Source: Apache Spark documentation - Cluster Mode Overview) This deployment is ideal for production environments where the job is submitted from a gateway node, and Spark manages the driver lifecycle on the cluster itself.

Option A is partially true but less specific than D.

Option B is incorrect: the driver never executes all tasks; executors handle distributed tasks.

Option C describes client mode, not cluster mode.

QUESTION NO: 50

Given the schema:

```
event_ts TIMESTAMP,  
sensor_id STRING,  
metric_value LONG,  
ingest_ts TIMESTAMP,  
source_file_path STRING
```

```
event_ts TIMESTAMP,  
sensor_id STRING,  
metric_value LONG,  
ingest_ts TIMESTAMP,
```

source_file_path STRING

The goal is to deduplicate based on: event_ts, sensor_id, and metric_value.

Options:

- A. dropDuplicates on all columns (wrong criteria)
- B. dropDuplicates with no arguments (removes based on all columns)
- C. groupBy without aggregation (invalid use)
- D. dropDuplicates on the exact matching fields

Answer: D

Explanation:

```
dedup_df = iot_bronze_df.dropDuplicates(["event_ts", "sensor_id", "metric_value"])
```

dropDuplicates accepts a list of columns to use for deduplication.

This ensures only unique records based on the specified keys are retained.

QUESTION NO: 51

A data scientist has identified that some records in the user profile table contain null values in any of the fields, and such records should be removed from the dataset before processing.

The schema includes fields like user_id, username, date_of_birth, created_ts, etc.

The schema of the user profile table looks like this:

```
user_id STRING,
username STRING,
full_name STRING,
date_of_birth DATE,
primary_email STRING,
created_ts TIMESTAMP,
updated_ts TIMESTAMP,
last_login_ts TIMESTAMP
```

Which block of Spark code can be used to achieve this requirement?

Options:

- A. filtered_df = users_raw_df.na.drop(thresh=0)
- B. filtered_df = users_raw_df.na.drop(how='all')
- C. filtered_df = users_raw_df.na.drop(how='any')
- D. filtered_df = users_raw_df.na.drop(how='all', thresh=None)

Answer: C

Explanation:

.na.drop(how='any') drops any row that has at least one null value.

This is exactly what's needed when the goal is to retain only fully complete records.

Usage: CopyEdit

```
filtered_df = users_raw_df.na.drop(how='any')
```

Explanation of incorrect options:

A: thresh=0 is invalid - thresh must be ≥ 1 .

B: how='all' drops only rows where all columns are null (too lenient).

D: spark.na.drop doesn't support mixing how and thresh in that way; it's incorrect syntax.

QUESTION NO: 52

A data engineer needs to write a DataFrame df to a Parquet file, partitioned by the column country, and overwrite any existing data at the destination path.

Which code should the data engineer use to accomplish this task in Apache Spark?

A. df.write.mode("overwrite").partitionBy("country").parquet("/data/output")

B. df.write.mode("append").partitionBy("country").parquet("/data/output")

C. df.write.mode("overwrite").parquet("/data/output")

D. df.write.partitionBy("country").parquet("/data/output")

Answer: A

Explanation:

The .mode("overwrite") ensures that existing files at the path will be replaced.

.partitionBy("country") optimizes queries by writing data into partitioned folders.

Correct syntax:

df.write.mode("overwrite").partitionBy("country").parquet("/data/output")

- Source: Spark SQL, DataFrames and Datasets Guide

QUESTION NO: 53

41 of 55.

A data engineer is working on the DataFrame df1 and wants the Name with the highest count to appear first (descending order by count), followed by the next highest, and so on.

The DataFrame has columns:

id | Name | count | timestamp

1 | USA | 10

2 | India | 20

3 | England | 50

4 | India | 50

5 | France | 20

6 | India | 10

7 | USA | 30

8 | USA | 40

Which code fragment should the engineer use to sort the data in the Name and count columns?

A. df1.orderBy(col("count").desc(), col("Name").asc())

B. df1.sort("Name", "count")

C. df1.orderBy("Name", "count")

D. df1.orderBy(col("Name").desc(), col("count").asc())

Answer: A

Explanation:

To sort a Spark DataFrame by multiple columns, use .orderBy() (or .sort()) with column

expressions.

Correct syntax for descending and ascending mix:

```
from pyspark.sql.functions import col
df1.orderBy(col("count").desc(), col("Name").asc())
```

This sorts primarily by count in descending order and secondarily by Name in ascending order (alphabetically).

Why the other options are incorrect:

B/C: Default sort order is ascending; won't place highest counts first.

D: Reverses sorting logic - sorts Name descending, not required.

Reference:

PySpark DataFrame API - orderBy() and col() for sorting with direction.

Databricks Exam Guide (June 2025): Section "Using Spark DataFrame APIs" - sorting, ordering, and column expressions.

QUESTION NO: 54

37 of 55.

A data scientist is working with a Spark DataFrame called customerDF that contains customer information.

The DataFrame has a column named email with customer email addresses.

The data scientist needs to split this column into username and domain parts.

Which code snippet splits the email column into username and domain columns?

A.

```
customerDF = customerDF \
.withColumn("username", split(col("email"), "@").getItem(0)) \
.withColumn("domain", split(col("email"), "@").getItem(1))
```

B.

```
customerDF = customerDF.withColumn("username", regexp_replace(col("email"), "@", ""))
```

C.

```
customerDF = customerDF.select("email").alias("username", "domain")
```

D.

```
customerDF = customerDF.withColumn("domain", col("email").split("@")[1])
```

Answer: A

Explanation:

The split() function in PySpark splits strings into an array based on a given delimiter.

Then, getItem(index) extracts a specific element from the array.

Correct usage:

```
from pyspark.sql.functions import split, col
customerDF = customerDF \
.withColumn("username", split(col("email"), "@").getItem(0)) \
.withColumn("domain", split(col("email"), "@").getItem(1))
```

This creates two new columns derived from the email field:

"username" → text before @

"domain" → text after @

Why the other options are incorrect:

B: regexp_replace only replaces text; does not split into multiple columns.

C: `.select()` cannot alias multiple derived columns like this.

D: Column objects are not native Python strings; cannot use standard `.split()`.

Reference:

PySpark SQL Functions - `split()` and `getItem()`.

Databricks Exam Guide (June 2025): Section "Developing Apache Spark

DataFrame/DataSet API Applications" - manipulating and splitting column data.

QUESTION NO: 55

9 of 55.

Given the code fragment:

```
import pyspark.pandas as ps
```

```
pdf = ps.DataFrame(data)
```

Which method is used to convert a Pandas API on Spark DataFrame

(`pyspark.pandas.DataFrame`) into a standard PySpark DataFrame (`pyspark.sql.DataFrame`)?

A. `pdf.to_pandas()`

B. `pdf.to_spark()`

C. `pdf.to_dataframe()`

D. `pdf.spark()`

Answer: B

Explanation:

In Pandas API on Spark (previously Koalas), the method `.to_spark()` converts a `pyspark.pandas.DataFrame` into a PySpark DataFrame.

Correct usage:

```
spark_df = pdf.to_spark()
```

This enables interoperability between the Pandas API on Spark and the PySpark SQL API, allowing developers to switch seamlessly between both for transformations or performance optimization.

Why the other options are incorrect:

A (`to_pandas()`): Converts to a local Pandas DataFrame, not a PySpark DataFrame.

C (`to_dataframe()`): Not a valid API method.

D (`spark()`): Not an existing DataFrame method.

Reference:

PySpark Pandas API Reference - `DataFrame.to_spark()` method.

Databricks Exam Guide (June 2025): Section "Using Pandas API on Apache Spark" - covers DataFrame conversions and interoperability.

QUESTION NO: 56

What is the difference between `df.cache()` and `df.persist()` in Spark DataFrame?

A. Both `cache()` and `persist()` can be used to set the default storage level (`MEMORY_AND_DISK_SER`)

B. Both functions perform the same operation. The `persist()` function provides improved performance as its default storage level is `DISK_ONLY`.

C. `persist()` - Persists the DataFrame with the default storage level (`MEMORY_AND_DISK_SER`) and `cache()` - Can be used to set different storage levels to persist the contents of the DataFrame.

D. cache() - Persists the DataFrame with the default storage level (MEMORY_AND_DISK) and **persist()** - Can be used to set different storage levels to persist the contents of the DataFrame

Answer: D

Explanation:

`df.cache()` is shorthand for `df.persist(StorageLevel.MEMORY_AND_DISK)`

`df.persist()` allows specifying any storage level such as `MEMORY_ONLY`, `DISK_ONLY`, `MEMORY_AND_DISK_SER`, etc.

By default, `persist()` uses `MEMORY_AND_DISK`, unless specified otherwise.

QUESTION NO: 57

A developer is running Spark SQL queries and notices underutilization of resources.

Executors are idle, and the number of tasks per stage is low.

What should the developer do to improve cluster utilization?

- A.** Increase the value of `spark.sql.shuffle.partitions`
- B.** Reduce the value of `spark.sql.shuffle.partitions`
- C.** Increase the size of the dataset to create more partitions
- D.** Enable dynamic resource allocation to scale resources as needed

Answer: A

Explanation:

The number of tasks is controlled by the number of partitions. By default, `spark.sql.shuffle.partitions` is 200. If stages are showing very few tasks (less than total cores), you may not be leveraging full parallelism.

From the Spark tuning guide:

"To improve performance, especially for large clusters, increase `spark.sql.shuffle.partitions` to create more tasks and parallelism." Thus:

A is correct: increasing shuffle partitions increases parallelism

B is wrong: it further reduces parallelism

C is invalid: increasing dataset size doesn't guarantee more partitions D is irrelevant to task count per stage Final answer: A

QUESTION NO: 58

A data analyst builds a Spark application to analyze finance data and performs the following operations: `filter`, `select`, `groupBy`, and `coalesce`.

Which operation results in a shuffle?

- A.** `groupBy`
- B.** `filter`
- C.** `select`
- D.** `coalesce`

Answer: A

Explanation:

The `groupBy()` operation causes a shuffle because it requires all values for a specific key to be brought together, which may involve moving data across partitions.

In contrast:

`filter()` and `select()` are narrow transformations and do not cause shuffles.
`coalesce()` tries to reduce the number of partitions and avoids shuffling by moving data to fewer partitions without a full shuffle (unlike `repartition()`).

QUESTION NO: 59

A data scientist of an e-commerce company is working with user data obtained from its subscriber database and has stored the data in a DataFrame `df_user`. Before further processing the data, the data scientist wants to create another DataFrame `df_user_non_pii` and store only the non-PII columns in this DataFrame. The PII columns in `df_user` are `first_name`, `last_name`, `email`, and `birthdate`.

Which code snippet can be used to meet this requirement?

- A. `df_user_non_pii = df_user.drop("first_name", "last_name", "email", "birthdate")`
- B. `df_user_non_pii = df_user.drop("first_name", "last_name", "email", "birthdate")`
- C. `df_user_non_pii = df_user.dropfields("first_name", "last_name", "email", "birthdate")`
- D. `df_user_non_pii = df_user.dropfields("first_name, last_name, email, birthdate")`

Answer: A

Explanation:

To remove specific columns from a PySpark DataFrame, the `drop()` method is used. This method returns a new DataFrame without the specified columns. The correct syntax for dropping multiple columns is to pass each column name as a separate argument to the `drop()` method.

Correct Usage:

```
df_user_non_pii = df_user.drop("first_name", "last_name", "email", "birthdate")
```

This line of code will return a new DataFrame `df_user_non_pii` that excludes the specified PII columns.

Explanation of Options:

- A . Correct. Uses the `drop()` method with multiple column names passed as separate arguments, which is the standard and correct usage in PySpark.
- B . Although it appears similar to Option A, if the column names are not enclosed in quotes or if there's a syntax error (e.g., missing quotes or incorrect variable names), it would result in an error. However, as written, it's identical to Option A and thus also correct.
- C . Incorrect. The `dropfields()` method is not a method of the DataFrame class in PySpark. It's used with StructType columns to drop fields from nested structures, not top-level DataFrame columns.
- D . Incorrect. Passing a single string with comma-separated column names to `dropfields()` is not valid syntax in PySpark.

Reference:

PySpark Documentation: `DataFrame.drop`

Stack Overflow Discussion: How to delete columns in PySpark DataFrame

QUESTION NO: 60

23 of 55.

A data scientist is working with a massive dataset that exceeds the memory capacity of a single machine. The data scientist is considering using Apache Spark™ instead of traditional single-machine languages like standard Python scripts.

Which two advantages does Apache Spark™ offer over a normal single-machine language in

this scenario? (Choose 2 answers)

- A.** It can distribute data processing tasks across a cluster of machines, enabling horizontal scalability.
- B.** It requires specialized hardware to run, making it unsuitable for commodity hardware clusters.
- C.** It processes data solely on disk storage, reducing the need for memory resources.
- D.** It eliminates the need to write any code, automatically handling all data processing.
- E.** It has built-in fault tolerance, allowing it to recover seamlessly from node failures during computation.

Answer: A,E

Explanation:

Apache Spark is a distributed data processing engine designed for large-scale, cluster-based computation.

Advantages:

Horizontal Scalability: Spark can distribute tasks across many machines, handling datasets larger than the memory of a single node.

Fault Tolerance: Spark automatically recovers from node or task failures using the lineage graph (RDD recovery mechanism) and retry logic.

These two features allow Spark to process huge datasets efficiently and reliably, unlike standard Python scripts that are limited to one machine and fail on single-node errors.

Why the other options are incorrect:

B: Spark runs on commodity hardware; no specialized machines required.

C: Spark emphasizes in-memory processing, not disk-only operations.

D: Spark still requires user code in Python, Scala, SQL, or Java.

Reference:

Databricks Exam Guide (June 2025): Section "Apache Spark Architecture and Components" - advantages, cluster execution, and fault tolerance.

Apache Spark Overview - distributed processing and resilience design.

QUESTION NO: 61

24 of 55.

Which code should be used to display the schema of the Parquet file stored in the location events.parquet?

A.

```
spark.sql("SELECT * FROM events.parquet").show()
```

B.

```
spark.read.format("parquet").load("events.parquet").show()
```

C.

```
spark.read.parquet("events.parquet").printSchema()
```

D.

```
spark.sql("SELECT schema FROM events.parquet").show()
```

Answer: C

Explanation:

To view the schema of a Parquet file, you must use the DataFrameReader to load the

Parquet data and call the `.printSchema()` method.

Correct syntax:

```
spark.read.parquet("events.parquet").printSchema()
```

This command loads the file metadata (without triggering a full read) and prints the column names, data types, and nullability information in a tree format.

Why the other options are incorrect:

A/D: SQL queries can't directly introspect file schemas.

B: `.show()` displays data rows, not schema.

Reference:

PySpark DataFrameReader API - `read.parquet()` and `DataFrame.printSchema()`.

Databricks Exam Guide (June 2025): Section "Using Spark SQL" - describes reading files and examining schemas in Spark SQL and DataFrame APIs.

QUESTION NO: 62

A data engineer is asked to build an ingestion pipeline for a set of Parquet files delivered by an upstream team on a nightly basis. The data is stored in a directory structure with a base path of `/path/events/data`. The upstream team drops daily data into the underlying subdirectories following the convention `year/month/day`.

A few examples of the directory structure are:

```
/path/events/data/2024/01/01
/path/events/data/2024/01/02
/path/events/data/2024/01/03
/path/events/data/2023/01/01
/path/events/data/2023/01/02
/path/events/data/2023/01/03
```

Which of the following code snippets will read all the data within the directory structure?

- A. `df = spark.read.option("inferSchema", "true").parquet("/path/events/data/")`
- B. `df = spark.read.option("recursiveFileLookup", "true").parquet("/path/events/data/")`
- C. `df = spark.read.parquet("/path/events/data/*")`
- D. `df = spark.read.parquet("/path/events/data/")`

Answer: B

Explanation:

To read all files recursively within a nested directory structure, Spark requires the `recursiveFileLookup` option to be explicitly enabled. According to Databricks official documentation, when dealing with deeply nested Parquet files in a directory tree (as shown in this example), you should set:

```
df = spark.read.option("recursiveFileLookup", "true").parquet("/path/events/data/")
```

This ensures that Spark searches through all subdirectories under `/path/events/data/` and reads any Parquet files it finds, regardless of the folder depth.

Option A is incorrect because while it includes an option, `inferSchema` is irrelevant here and does not enable recursive file reading.

Option C is incorrect because wildcards may not reliably match deep nested structures beyond one directory level.

Option D is incorrect because it will only read files directly within /path/events/data/ and not subdirectories like /2023/01/01.

Databricks documentation reference:

"To read files recursively from nested folders, set the recursiveFileLookup option to true. This is useful when data is organized in hierarchical folder structures" - Databricks documentation on Parquet files ingestion and options.

QUESTION NO: 63

Given this code:

```
inputStream
  .withWatermark("event_time", "10 minutes")
  .groupBy(window("event_time", "15 minutes"))
  .count()
```

```
.withWatermark("event_time", "10 minutes")
.groupBy(window("event_time", "15 minutes"))
.count()
```

What happens to data that arrives after the watermark threshold?

Options:

- A. Records that arrive later than the watermark threshold (10 minutes) will automatically be included in the aggregation if they fall within the 15-minute window.
- B. Any data arriving more than 10 minutes after the watermark threshold will be ignored and not included in the aggregation.
- C. Data arriving more than 10 minutes after the latest watermark will still be included in the aggregation but will be placed into the next window.
- D. The watermark ensures that late data arriving within 10 minutes of the latest event_time will be processed and included in the windowed aggregation.

Answer: B

Explanation:

According to Spark's watermarking rules:

"Records that are older than the watermark (event time < current watermark) are considered too late and are dropped." So, if a record's event_time is earlier than (max event_time seen so far - 10 minutes), it is discarded.

QUESTION NO: 64

10 of 55.

What is the benefit of using Pandas API on Spark for data transformations?

- A. It executes queries faster using all the available cores in the cluster as well as provides Pandas's rich set of features.
- B. It is available only with Python, thereby reducing the learning curve.
- C. It runs on a single node only, utilizing memory efficiently.

D. It computes results immediately using eager execution.

Answer: A

Explanation:

Pandas API on Spark provides a distributed implementation of the Pandas DataFrame API on top of Apache Spark.

Advantages:

Executes transformations in parallel across all nodes and cores in the cluster.

Maintains Pandas-like syntax, making it easy for Python users to transition.

Enables scaling of existing Pandas code to handle large datasets without memory limits.

Therefore, it combines Pandas usability with Spark's distributed power, offering both speed and scalability.

Why the other options are incorrect:

B: While it uses Python, that's not its main advantage.

C: It runs distributed across the cluster, not on a single node.

D: Pandas API on Spark uses lazy evaluation, not eager computation.

Reference:

PySpark Pandas API Overview - advantages of distributed execution.

Databricks Exam Guide (June 2025): Section "Using Pandas API on Apache Spark" - explains the benefits of Pandas API integration for scalable transformations.

QUESTION NO: 65

36 of 55.

What is the main advantage of partitioning the data when persisting tables?

A. It compresses the data to save disk space.

B. It automatically cleans up unused partitions to optimize storage.

C. It ensures that data is loaded into memory all at once for faster query execution.

D. It optimizes by reading only the relevant subset of data from fewer partitions.

Answer: D

Explanation:

Partitioning a dataset divides data into separate directories based on partition column values. When queries filter on partitioned columns, Spark can prune irrelevant partitions - meaning it only reads files that match the filter criteria.

Advantage:

Reduces I/O and improves performance by scanning only relevant subsets of data.

Example:

/data/sales/year=2023/month=10/...

/data/sales/year=2024/month=01/...

A query filtering WHERE year = 2024 reads only the relevant partition.

Why the other options are incorrect:

A: Compression is independent of partitioning.

B: Spark does not automatically clean partitions unless managed manually.

C: Partitioning does not cause Spark to load entire data into memory.

Reference:

Databricks Exam Guide (June 2025): Section "Using Spark SQL" - partitioning and pruning for optimized data retrieval.

Spark SQL Documentation - DataFrameWriter partitionBy() and query optimization.

QUESTION NO: 66

A data engineer is running a Spark job to process a dataset of 1 TB stored in distributed storage. The cluster has 10 nodes, each with 16 CPUs. Spark UI shows:

Low number of Active Tasks

Many tasks complete in milliseconds

Fewer tasks than available CPUs

Which approach should be used to adjust the partitioning for optimal resource allocation?

- A. Set the number of partitions equal to the total number of CPUs in the cluster
- B. Set the number of partitions to a fixed value, such as 200
- C. Set the number of partitions equal to the number of nodes in the cluster
- D. Set the number of partitions by dividing the dataset size (1 TB) by a reasonable partition size, such as 128 MB

Answer: D

Explanation:

Spark's best practice is to estimate partition count based on data volume and a reasonable partition size - typically 128 MB to 256 MB per partition.

With 1 TB of data: $1 \text{ TB} / 128 \text{ MB} \approx \sim 8000$ partitions

This ensures that tasks are distributed across available CPUs for parallelism and that each task processes an optimal volume of data.

Option A (equal to cores) may result in partitions that are too large.

Option B (fixed 200) is arbitrary and may underutilize the cluster.

Option C (nodes) gives too few partitions (10), limiting parallelism.

QUESTION NO: 67

A developer wants to refactor some older Spark code to leverage built-in functions introduced in Spark 3.5.0. The existing code performs array manipulations manually. Which of the following code snippets utilizes new built-in functions in Spark 3.5.0 for array operations?

```
import pyspark.sql.functions as F

min_price = 110.50

result_df = prices_df \
    .filter(F.col("spot_price") >= F.lit(min_price)) \
    .agg(F.count("*"))
```

A.

```
result_df = prices_df \
    .withColumn("valid_price", F.when(F.col("spot_price") > F.lit(min_price), 1).otherwise(0))
```

```
result_df = prices_df \
    .withColumn("valid_price", F.when(F.col("spot_price") > F.lit(min_price), 1).otherwise(0))
```

B.

```
result_df = prices_df \
    .agg(F.count_if(F.col("spot_price") >= F.lit(min_price)))
```

```
result_df = prices_df \
    .agg(F.count_if(F.col("spot_price") >= F.lit(min_price)))
```

C.

```
result_df = prices_df \
    .agg(F.min("spot_price"), F.max("spot_price"))
```

```
result_df = prices_df \
    .agg(F.min("spot_price"), F.max("spot_price"))
```

D.

```
result_df = prices_df \
    .agg(F.count("spot_price").alias("spot_price")) \
    .filter(F.col("spot_price") > F.lit("min_price"))
```

```
result_df = prices_df \
    .agg(F.count("spot_price").alias("spot_price")) \
    .filter(F.col("spot_price") > F.lit("min_price"))
```

Answer: B

Explanation:

count_if(condition) counts the number of rows that meet the specified boolean condition.

In this example, it directly counts how many times spot_price >= min_price evaluates to true, replacing the older verbose combination of when/otherwise and filtering or summing.

Official Spark 3.5.0 documentation notes the addition of count_if to simplify this kind of logic:

"Added count_if aggregate function to count only the rows where a boolean condition holds (SPARK-43773)." Why other options are incorrect or outdated:

A uses a legacy-style method of adding a flag column (when().otherwise()), which is verbose compared to count_if.

C performs a simple min/max aggregation-useful but unrelated to conditional array operations or the updated functionality.

D incorrectly applies .filter() after .agg() which will cause an error, and misuses string "min_price" rather than the variable.

Therefore, B is the only option leveraging new functionality from Spark 3.5.0 correctly and efficiently.

Explanation:

The correct answer is B because it uses the new function count_if, introduced in Spark 3.5.0, which simplifies conditional counting within aggregations.

QUESTION NO: 68

A data engineer is working on the DataFrame:

Id	Name	count	timestamp
4	Washington	10	2024-09-19T10:10:40.000+00:00
1	Delhi	20	2024-09-19T10:10:10.000+00:00
2	London	50	2024-09-19T10:10:20.000+00:00
1	Delhi	50	2024-09-19T10:10:50.000+00:00
3	Paris	20	2024-09-19T10:11:20.000+00:00
1	Delhi	10	2024-09-19T10:11:10.000+00:00
3	Paris	30	2024-09-19T10:10:30.000+00:00
4	Washington	40	2024-09-19T10:11:00.000+00:00

(Referring to the table image: it has columns Id, Name, count, and timestamp.) Which code fragment should the engineer use to extract the unique values in the Name column into an alphabetically ordered list?

- A. `df.select("Name").orderBy(df["Name"].asc())`
- B. `df.select("Name").distinct().orderBy(df["Name"])`
- C. `df.select("Name").distinct()`
- D. `df.select("Name").distinct().orderBy(df["Name"].desc())`

Answer: B

Explanation:

To extract unique values from a column and sort them alphabetically:

`distinct()` is required to remove duplicate values.

`orderBy()` is needed to sort the results alphabetically (ascending by default).

Correct code:

`df.select("Name").distinct().orderBy(df["Name"])`

This is directly aligned with standard DataFrame API usage in PySpark, as documented in the official Databricks Spark APIs. Option A is incorrect because it may not remove duplicates. Option C omits sorting. Option D sorts in descending order, which doesn't meet the requirement for alphabetical (ascending) order.

QUESTION NO: 69

A DataFrame `df` has columns `name`, `age`, and `salary`. The developer needs to sort the DataFrame by `age` in ascending order and `salary` in descending order.

Which code snippet meets the requirement of the developer?

- A. `df.orderBy(col("age").asc(), col("salary").asc()).show()`
- B. `df.sort("age", "salary", ascending=[True, True]).show()`
- C. `df.sort("age", "salary", ascending=[False, True]).show()`
- D. `df.orderBy("age", "salary", ascending=[True, False]).show()`

Answer: D

Explanation:

To sort a PySpark DataFrame by multiple columns with mixed sort directions, the correct usage is:

python

CopyEdit

```
df.orderBy("age", "salary", ascending=[True, False])
```

age will be sorted in ascending order

salary will be sorted in descending order

The orderBy() and sort() methods in PySpark accept a list of booleans to specify the sort direction for each column.

Documentation Reference: PySpark API - DataFrame.orderBy

QUESTION NO: 70

2 of 55. Which command overwrites an existing JSON file when writing a DataFrame?

- A. df.write.json("path/to/file")
- B. df.write.mode("append").json("path/to/file")
- C. df.write.option("overwrite").json("path/to/file")
- D. df.write.mode("overwrite").json("path/to/file")

Answer: D

Explanation:

When writing DataFrames to files using the Spark DataFrameWriter API, Spark by default raises an error if the target path already exists. To explicitly overwrite existing data, you must specify the write mode as "overwrite".

Correct Syntax:

```
df.write.mode("overwrite").json("path/to/file")
```

This command removes the existing file or directory at the specified path and writes the new output in JSON format.

Other supported save modes include:

"append" - Adds new data to existing files.

"ignore" - Skips writing if the path already exists.

"error" or "errorifexists" - Fails the job if the output path exists (default).

Why other options are incorrect:

A: Defaults to "error" mode, which fails if the path exists.

B: "append" only adds data; it does not overwrite existing data.

C: .option("overwrite") is invalid - mode("overwrite") must be used instead.

Reference (Databricks Apache Spark 3.5 - Python / Study Guide):

PySpark API Reference: DataFrameWriter.mode() - describes valid write modes including "overwrite".

PySpark API Reference: DataFrameWriter.json() - method to write DataFrames in JSON format.

Databricks Certified Associate Developer for Apache Spark Exam Guide (June 2025):

Section "Using Spark DataFrame APIs" - Reading and writing DataFrames using save modes, schema management, and partitioning.

QUESTION NO: 71

A data engineer wants to process a streaming DataFrame that receives sensor readings every second with columns `sensor_id`, `temperature`, and `timestamp`. The engineer needs to calculate the average temperature for each sensor over the last 5 minutes while the data is streaming.

Which code implementation achieves the requirement?

Options from the images provided:

A.

```
df.withColumn("avg_temp", avg("temperature")
    .over(Window.partitionBy("sensor_id"))
```

B.

```
df.groupBy("sensor_id", "timestamp")
    .agg(avg("temperature").alias("avg_temp"))
```

C.

```
df.groupBy("sensor_id").avg("temperature")
df.withWatermark("timestamp", "5 minutes")
```

D.

```
.groupBy("sensor_id", window("timestamp", "5 minutes"))
.agg(avg("temperature").alias("avg_temp"))
```

Answer: D

Explanation:

The correct answer is D because it uses proper time-based window aggregation along with watermarking, which is the required pattern in Spark Structured Streaming for time-based aggregations over event-time data.

From the Spark 3.5 documentation on structured streaming:

"You can define sliding windows on event-time columns, and use `groupBy` along with `window()` to compute aggregates over those windows. To deal with late data, you use `withWatermark()` to specify how late data is allowed to arrive." (Source: Structured Streaming Programming Guide) In option D, the use of:

python

CopyEdit

```
.groupBy("sensor_id", window("timestamp", "5 minutes"))
.agg(avg("temperature").alias("avg_temp"))
```

ensures that for each `sensor_id`, the average temperature is calculated over 5-minute event-time windows. To complete the logic, it is assumed that `withWatermark("timestamp", "5 minutes")` is used earlier in the pipeline to handle late events.

Explanation of why other options are incorrect:

Option A uses `Window.partitionBy` which applies to static DataFrames or batch queries and is not suitable for streaming aggregations.

Option B does not apply a time window, thus does not compute the rolling average over 5 minutes.

Option C incorrectly applies `withWatermark()` after an aggregation and does not include any

time window, thus missing the time-based grouping required.

Therefore, Option D is the only one that meets all requirements for computing a time-windowed streaming aggregation.

QUESTION NO: 72

7 of 55.

A developer has been asked to debug an issue with a Spark application. The developer identified that the data being loaded from a CSV file is being read incorrectly into a DataFrame.

The CSV file has been read using the following Spark SQL statement:

```
CREATE TABLE locations
```

```
USING csv
```

```
OPTIONS (path '/data/locations.csv')
```

The first lines of the command `SELECT * FROM locations` look like this:

```
| city | lat | long |
```

```
| ALTI Sydney | -33... | ... |
```

Which parameter can the developer add to the `OPTIONS` clause in the `CREATE TABLE` statement to read the CSV data correctly again?

- A. `'header' 'true'`
- B. `'header' 'false'`
- C. `'sep' ','`
- D. `'sep' '|'`

Answer: A

Explanation:

When reading CSV files using Spark SQL or the DataFrame API, Spark by default assumes that the first line of the file is data, not headers. To interpret the first line as column names, the header option must be set to true.

Correct syntax:

```
CREATE TABLE locations
```

```
USING csv
```

```
OPTIONS (
```

```
path '/data/locations.csv',
```

```
header 'true'
```

```
);
```

This tells Spark to read the first row as column headers and correctly map columns like city, lat, and long.

Why the other options are incorrect:

B (header 'false'): Default behavior; would keep reading header as data.

C / D (sep): Used to specify the delimiter; not relevant unless the file uses a different separator (e.g., |).

Reference (Databricks Apache Spark 3.5 - Python / Study Guide):

PySpark SQL Data Sources - CSV options (header, inferSchema, sep).

Databricks Exam Guide (June 2025): Section "Using Spark SQL" - Reading data from files with different formats using Spark SQL and DataFrame APIs.

QUESTION NO: 73

4 of 55.

A developer is working on a Spark application that processes a large dataset using SQL queries. Despite having a large cluster, the developer notices that the job is underutilizing the available resources. Executors remain idle for most of the time, and logs reveal that the number of tasks per stage is very low. The developer suspects that this is causing suboptimal cluster performance.

Which action should the developer take to improve cluster utilization?

- A. Increase the value of `spark.sql.shuffle.partitions`
- B. Reduce the value of `spark.sql.shuffle.partitions`
- C. Enable dynamic resource allocation to scale resources as needed
- D. Increase the size of the dataset to create more partitions

Answer: A

Explanation:

In Spark SQL and DataFrame operations, the configuration parameter `spark.sql.shuffle.partitions` defines the number of partitions created during shuffle operations such as `join`, `groupBy`, and `distinct`.

The default value (in Spark 3.5) is 200.

If this number is too low, Spark creates fewer tasks, leading to idle executors and poor cluster utilization.

Increasing this value allows Spark to create more tasks that can run in parallel across executors, effectively using more cluster resources.

Correct approach:

```
spark.conf.set("spark.sql.shuffle.partitions", 400)
```

This increases the parallelism level of shuffle stages and improves overall resource utilization.

Why the other options are incorrect:

B: Reducing partitions further would decrease parallelism and worsen the underutilization issue.

C: Dynamic resource allocation scales executors up or down based on workload, but it doesn't fix low task parallelism caused by insufficient shuffle partitions.

D: Increasing dataset size is not a tuning solution and doesn't address task-level under-parallelization.

Reference (Databricks Apache Spark 3.5 - Python / Study Guide):

Spark SQL Configuration: `spark.sql.shuffle.partitions` - controls the number of shuffle partitions.

Databricks Exam Guide (June 2025): Section "Troubleshooting and Tuning Apache Spark DataFrame API Applications" - tuning strategies, partitioning, and optimizing cluster utilization

QUESTION NO: 74

Which UDF implementation calculates the length of strings in a Spark DataFrame?

- A. `df.withColumn("length", spark.udf("len", StringType()))`
- B. `df.select(length(col("stringColumn")).alias("length"))`
- C. `spark.udf.register("stringLength", lambda s: len(s))`

D. `df.withColumn("length", udf(lambda s: len(s), StringType()))`

Answer: B

Explanation:

Option B uses Spark's built-in SQL function `length()`, which is efficient and avoids the overhead of a Python UDF:

```
from pyspark.sql.functions import length, col
df.select(length(col("stringColumn")).alias("length"))
```

Explanation of other options:

Option A is incorrect syntax; `spark.udf` is not called this way.

Option C registers a UDF but doesn't apply it in the DataFrame transformation.

Option D is syntactically valid but uses a Python UDF which is less efficient than built-in functions.

Final answer: B

QUESTION NO: 75

A data engineer uses a broadcast variable to share a DataFrame containing millions of rows across executors for lookup purposes. What will be the outcome?

- A.** The job may fail if the memory on each executor is not large enough to accommodate the DataFrame being broadcasted
- B.** The job may fail if the executors do not have enough CPU cores to process the broadcasted dataset
- C.** The job will hang indefinitely as Spark will struggle to distribute and serialize such a large broadcast variable to all executors
- D.** The job may fail because the driver does not have enough CPU cores to serialize the large DataFrame

Answer: A

Explanation:

In Apache Spark, broadcast variables are used to efficiently distribute large, read-only data to all worker nodes. However, broadcasting very large datasets can lead to memory issues on executors if the data does not fit into the available memory.

According to the Spark documentation:

"Broadcast variables allow the programmer to keep a read-only variable cached on each machine rather than shipping a copy of it with tasks. This can greatly reduce the amount of data sent over the network." However, it also notes:

"Using the broadcast functionality available in `SparkContext` can greatly reduce the size of each serialized task, and the cost of launching a job over a cluster. If your tasks use any large object from the driver program inside of them (e.g., a static lookup table), consider turning it into a broadcast variable." But caution is advised when broadcasting large datasets: "Broadcasting large variables can cause out-of-memory errors if the data does not fit in the memory of each executor." Therefore, if the broadcasted DataFrame containing millions of rows exceeds the memory capacity of the executors, the job may fail due to memory constraints.

QUESTION NO: 76

A data engineer noticed improved performance after upgrading from Spark 3.0 to Spark 3.5.

The engineer found that Adaptive Query Execution (AQE) was enabled.

Which operation is AQE implementing to improve performance?

- A. Dynamically switching join strategies
- B. Collecting persistent table statistics and storing them in the metastore for future use
- C. Improving the performance of single-stage Spark jobs
- D. Optimizing the layout of Delta files on disk

Answer: A

Explanation:

Adaptive Query Execution (AQE) is a Spark 3.x feature that dynamically optimizes query plans at runtime. One of its core features is:

Dynamically switching join strategies (e.g., from sort-merge to broadcast) based on runtime statistics.

Other AQE capabilities include:

Coalescing shuffle partitions

Skew join handling

Option A is correct.

Option B refers to statistics collection, which is not AQE's primary function.

Option C is too broad and not AQE-specific.

Option D refers to Delta Lake optimizations, unrelated to AQE.

Final answer: A

QUESTION NO: 77

30 of 55.

A data engineer is working on a num_df DataFrame and has a Python UDF defined as:

```
def cube_func(val):
```

```
    return val * val * val
```

Which code fragment registers and uses this UDF as a Spark SQL function to work with the DataFrame num_df?

A.

```
spark.udf.register("cube_func", cube_func)
```

```
num_df.selectExpr("cube_func(num)").show()
```

B.

```
num_df.select(cube_func("num")).show()
```

C.

```
spark.createDataFrame(cube_func("num")).show()
```

D.

```
num_df.register("cube_func").select("num").show()
```

Answer: A

Explanation:

To use a Python function as a UDF (User Defined Function) in Spark SQL, it must first be registered using `spark.udf.register()`.

Correct usage:

```
spark.udf.register("cube_func", cube_func)
```

```
num_df.selectExpr("cube_func(num)").show()
```

This registers `cube_func` as a callable SQL function available in expressions or queries.

Why the other options are incorrect:

B: You must wrap with `udf()` or `selectExpr`; calling plain Python functions won't work.

C: `createDataFrame` is for building DataFrames, not calling UDFs.

D: DataFrames cannot directly register UDFs.

Reference:

PySpark SQL Functions - `spark.udf.register()` and `selectExpr()`.

Databricks Exam Guide (June 2025): Section "Using Spark SQL" - user-defined functions and Spark SQL integration.

QUESTION NO: 78

What is the benefit of using Pandas on Spark for data transformations?

Options:

A. It is available only with Python, thereby reducing the learning curve.

B. It computes results immediately using eager execution, making it simple to use.

C. It runs on a single node only, utilizing the memory with memory-bound DataFrames and hence cost-efficient.

D. It executes queries faster using all the available cores in the cluster as well as provides Pandas's rich set of features.

Answer: D

Explanation:

Pandas API on Spark (formerly Koalas) offers:

Familiar Pandas-like syntax

Distributed execution using Spark under the hood

Scalability for large datasets across the cluster

It provides the power of Spark while retaining the productivity of Pandas.

QUESTION NO: 79

3 of 55. A data engineer observes that the upstream streaming source feeds the event table frequently and sends duplicate records. Upon analyzing the current production table, the data engineer found that the time difference in the `event_timestamp` column of the duplicate records is, at most, 30 minutes.

To remove the duplicates, the engineer adds the code:

```
df = df.withWatermark("event_timestamp", "30 minutes")
```

What is the result?

A. It removes all duplicates regardless of when they arrive.

B. It accepts watermarks in seconds and the code results in an error.

C. It removes duplicates that arrive within the 30-minute window specified by the watermark.

D. It is not able to handle deduplication in this scenario.

Answer: C

Explanation:

In Structured Streaming, a watermark defines the maximum delay for event-time data to be considered in stateful operations like deduplication or window aggregations.

Behavior:

```
df = df.withWatermark("event_timestamp", "30 minutes")
```


This sets a 30-minute watermark, meaning Spark will only keep track of events that arrive within 30 minutes of the latest event time seen so far. When used with:

```
df.dropDuplicates(["event_id", "event_timestamp"])
```

Spark removes duplicates that arrive within the watermark threshold (in this case, within 30 minutes).

Why other options are incorrect:

A: Watermarks do not remove all duplicates; they only manage those within the defined event-time window.

B: Watermark durations can be expressed as strings like "30 minutes", "10 seconds", etc., not only seconds.

D: Structured Streaming supports deduplication using `withWatermark()` and `dropDuplicates()`.

Reference (Databricks Apache Spark 3.5 - Python / Study Guide):

PySpark Structured Streaming Guide - `withWatermark()` and `dropDuplicates()` methods for event-time deduplication.

Databricks Certified Associate Developer for Apache Spark Exam Guide (June 2025):

Section "Structured Streaming" - Topic: Streaming Deduplication with and without watermark usage.

QUESTION NO: 80

18 of 55.

An engineer has two DataFrames - df1 (small) and df2 (large). To optimize the join, the engineer uses a broadcast join:

```
from pyspark.sql.functions import broadcast
```

```
df_result = df2.join(broadcast(df1), on="id", how="inner")
```

What is the purpose of using `broadcast()` in this scenario?

A. It increases the partition size for df1 and df2.

B. It ensures that the join happens only when the id values are identical.

C. It reduces the number of shuffle operations by replicating the smaller DataFrame to all nodes.

D. It filters the id values before performing the join.

Answer: C

Explanation:

A broadcast join is a type of join where the smaller DataFrame is replicated (broadcast) to all worker nodes in the cluster. This avoids shuffling the large DataFrame across the network.

Benefits:

Eliminates shuffle for the smaller dataset.

Greatly improves performance when one side of the join is small enough to fit in memory.

Correct usage example:

```
df_result = df2.join(broadcast(df1), "id")
```

This is a map-side join, where each executor joins its local partition of the large dataset with the broadcasted copy of the small one.

Why the other options are incorrect:

A: Broadcasting does not change partition sizes.

B: Joins always match on key equality; this is not specific to broadcast joins.

D: Broadcasting does not filter; it distributes data for faster joins.

Reference:

Databricks Exam Guide (June 2025): Section "Developing Apache Spark DataFrame/DataSet API Applications" - broadcast joins and partitioning strategies.
PySpark SQL Functions - broadcast() method documentation.

QUESTION NO: 81

A data engineer replaces the exact percentile() function with approx_percentile() to improve performance, but the results are drifting too far from expected values.
Which change should be made to solve the issue?

```
features_df = raw_df \
    .select(F.approx_percentile("price", percentage=[0.25, 0.5, 0.75], accuracy=1000).alias("price_perc"))
```

- A. Decrease the first value of the percentage parameter to increase the accuracy of the percentile ranges
- B. Decrease the value of the accuracy parameter in order to decrease the memory usage but also improve the accuracy
- C. Increase the last value of the percentage parameter to increase the accuracy of the percentile ranges
- D. Increase the value of the accuracy parameter in order to increase the memory usage but also improve the accuracy

Answer: D

Explanation:

The approx_percentile function in Spark is a performance-optimized alternative to percentile. It takes an optional accuracy parameter:

approx_percentile(column, percentage, accuracy)

Higher accuracy values → more precise results, but increased memory/computation.

Lower values → faster but less accurate.

From the documentation:

"Increasing the accuracy improves precision but increases memory usage." Final answer: D

QUESTION NO: 82

11 of 55.

Which Spark configuration controls the number of tasks that can run in parallel on an executor?

- A. spark.executor.cores
- B. spark.task.maxFailures
- C. spark.executor.memory
- D. spark.sql.shuffle.partitions

Answer: A

Explanation:

The Spark configuration spark.executor.cores defines how many concurrent tasks can be executed within a single executor process.

Each executor is assigned a number of CPU cores.

Each core executes one task at a time.

Therefore, increasing spark.executor.cores allows an executor to run more tasks

concurrently.

Example:

```
--conf spark.executor.cores=4
```

→ Each executor can run 4 parallel tasks.

Why the other options are incorrect:

B (spark.task.maxFailures): Sets retry attempts for failed tasks.

C (spark.executor.memory): Sets executor memory, not concurrency.

D (spark.sql.shuffle.partitions): Defines number of shuffle partitions, not executor concurrency.

Reference:

Spark Configuration Guide - Executor cores, tasks, and parallelism.

Databricks Exam Guide (June 2025): Section "Apache Spark Architecture and Components" - executor configuration, CPU cores, and parallel task execution.

QUESTION NO: 83

A Spark developer wants to improve the performance of an existing PySpark UDF that runs a hash function that is not available in the standard Spark functions library. The existing UDF code is:

```
import hashlib

import pyspark.sql.functions as sf
from pyspark.sql.types import StringType

def shake_256(raw):
    return hashlib.shake_256(raw.encode()).hexdigest(20)

shake_256_udf = sf.udf(shake_256, StringType())
```

```
import hashlib
import pyspark.sql.functions as sf
from pyspark.sql.types import StringType
def shake_256(raw):
    return hashlib.shake_256(raw.encode()).hexdigest(20)
shake_256_udf = sf.udf(shake_256, StringType())
```

The developer wants to replace this existing UDF with a Pandas UDF to improve performance. The developer changes the definition of shake_256_udf to this: CopyEdit
shake_256_udf = sf.pandas_udf(shake_256, StringType()) However, the developer receives the error:

What should the signature of the shake_256() function be changed to in order to fix this error?

- A. def shake_256(df: pd.Series) -> str:
- B. def shake_256(df: Iterator[pd.Series]) -> Iterator[pd.Series]:
- C. def shake_256(raw: str) -> str:
- D. def shake_256(df: pd.Series) -> pd.Series:

Answer: D

Explanation:

When converting a standard PySpark UDF to a Pandas UDF for performance optimization, the function must operate on a Pandas Series as input and return a Pandas Series as output.

In this case, the original function signature:

```
def shake_256(raw: str) -> str
```

is scalar - not compatible with Pandas UDFs.

According to the official Spark documentation:

"Pandas UDFs operate on pandas.Series and return pandas.Series. The function definition should be:

```
def my_udf(s: pd.Series) -> pd.Series:
```

and it must be registered using pandas_udf(...)."

Therefore, to fix the error:

The function should be updated to:

```
def shake_256(df: pd.Series) -> pd.Series:
```

```
    return df.apply(lambda x: hashlib.shake_256(x.encode()).hexdigest(20))
```

This will allow Spark to efficiently execute the Pandas UDF in vectorized form, improving performance compared to standard UDFs.

QUESTION NO: 84

A data engineer is reviewing a Spark application that applies several transformations to a DataFrame but notices that the job does not start executing immediately.

Which two characteristics of Apache Spark's execution model explain this behavior?

Choose 2 answers:

- A. The Spark engine requires manual intervention to start executing transformations.
- B. Only actions trigger the execution of the transformation pipeline.
- C. Transformations are executed immediately to build the lineage graph.
- D. The Spark engine optimizes the execution plan during the transformations, causing delays.
- E. Transformations are evaluated lazily.

Answer: B,E

Explanation:

Apache Spark employs a lazy evaluation model for transformations. This means that when transformations (e.g., map(), filter()) are applied to a DataFrame, Spark does not execute them immediately. Instead, it builds a logical plan (lineage) of transformations to be applied. Execution is deferred until an action (e.g., collect(), count(), save()) is called. At that point, Spark's Catalyst optimizer analyzes the logical plan, optimizes it, and then executes the physical plan to produce the result.

This lazy evaluation strategy allows Spark to optimize the execution plan, minimize data shuffling, and improve overall performance by reducing unnecessary computations.

QUESTION NO: 85

20 of 55.

What is the difference between df.cache() and df.persist() in Spark DataFrame?

- A. Both functions perform the same operation. The persist() function provides improved

performance as its default storage level is DISK_ONLY.

B. `persist()` - Persists the DataFrame with the default storage level (MEMORY_AND_DISK_DESER), and `cache()` - Can be used to set different storage levels.

C. Both `cache()` and `persist()` can be used to set the default storage level (MEMORY_AND_DISK_DESER).

D. `cache()` - Persists the DataFrame with the default storage level (MEMORY_AND_DISK_DESER), and `persist()` - Can be used to set different storage levels to persist the contents of the DataFrame.

Answer: D

Explanation:

Both `cache()` and `persist()` are Spark DataFrame storage operations that store computed results in memory (and optionally on disk) to speed up subsequent actions on the same DataFrame.

Key difference:

`cache()` is a shorthand for `persist(StorageLevel.MEMORY_AND_DISK)`.

`persist()` allows specifying different storage levels, such as MEMORY_ONLY, DISK_ONLY, or MEMORY_AND_DISK_SER.

Example:

`df.cache()` # Uses MEMORY_AND_DISK by default

`df.persist(StorageLevel.MEMORY_ONLY)` # Custom storage level

Both trigger caching upon an action (e.g., `count()`, `collect()`).

Why the other options are incorrect:

A: `persist()` default is not DISK_ONLY; default storage level is MEMORY_AND_DISK.

B/C: `cache()` cannot set arbitrary levels; only `persist()` can.

Reference:

PySpark API Reference - `DataFrame.cache()` and `DataFrame.persist()`.

Databricks Exam Guide (June 2025): Section "Developing Apache Spark

`DataFrame/DataSet` API Applications" - caching, persistence, and storage levels.

QUESTION NO: 86

A Spark application is experiencing performance issues in client mode because the driver is resource-constrained.

How should this issue be resolved?

A. Add more executor instances to the cluster

B. Increase the driver memory on the client machine

C. Switch the deployment mode to cluster mode

D. Switch the deployment mode to local mode

Answer: C

Explanation:

In Spark's client mode, the driver runs on the local machine that submitted the job. If that machine is resource-constrained (e.g., low memory), performance degrades.

From the Spark documentation:

"In cluster mode, the driver runs inside the cluster, benefiting from cluster resources and scalability." Option A is incorrect - executors do not help the driver directly.

Option B might help short-term but does not scale.

Option C is correct - switching to cluster mode moves the driver to the cluster.

Option D (local mode) is for development/testing, not production.

Final answer: C

QUESTION NO: 87

Which command overwrites an existing JSON file when writing a DataFrame?

- A. `df.write.mode("overwrite").json("path/to/file")`
- B. `df.write.overwrite.json("path/to/file")`
- C. `df.write.json("path/to/file", overwrite=True)`
- D. `df.write.format("json").save("path/to/file", mode="overwrite")`

Answer: A

Explanation:

The correct way to overwrite an existing file using the DataFrameWriter is:

`df.write.mode("overwrite").json("path/to/file")`

Option D is also technically valid, but Option A is the most concise and idiomatic PySpark syntax.

QUESTION NO: 88

13 of 55.

A developer needs to produce a Python dictionary using data stored in a small Parquet table, which looks like this:

region_id	region_name
10	North
12	East
14	West

The resulting Python dictionary must contain a mapping of region_id to region_name, containing the smallest 3 region_id values.

Which code fragment meets the requirements?

- A. `regions_dict = dict(regions.take(3))`
- B. `regions_dict = regions.select("region_id", "region_name").take(3)`
- C. `regions_dict = dict(regions.select("region_id", "region_name").rdd.collect())`
- D. `regions_dict = dict(regions.orderBy("region_id").limit(3).rdd.map(lambda x: (x.region_id, x.region_name)).collect())`

Answer: D

Explanation:

To create a Python dictionary from a Spark DataFrame, you can first collect the data to the driver node and then convert it into a Python dictionary using `dict()`.

Steps:

Select only relevant columns.

Order by region_id to get the smallest ones.

Limit to 3 rows.

Map each row into key-value pairs.

Collect results to the driver and convert to a dictionary.

Correct code:

```
regions_dict = dict(
regions.orderBy("region_id")
.limit(3)
.rdd.map(lambda x: (x.region_id, x.region_name))
.collect()
)
```

This produces a dictionary like:

```
{10: 'North', 12: 'East', 14: 'West'}
```

Why the other options are incorrect:

A/B: take(3) returns a list of Row objects, not key-value pairs.

C: Doesn't order or limit by smallest IDs, so the result may not be correct.

Reference:

PySpark RDD API - map() and collect().

Databricks Exam Guide (June 2025): Section "Using Spark DataFrame APIs" - covers DataFrame-to-local data conversions and collect operations.

QUESTION NO: 89

A developer is working with a pandas DataFrame containing user behavior data from a web application.

Which approach should be used for executing a groupBy operation in parallel across all workers in Apache Spark 3.5?

A)

Use the applyInPandas API

B)

```
import pandas as pd
def mean_func(key, pdf):
    return pd.DataFrame([key + (pdf['value'].mean(),)])
```

```
df.groupby('user_id').applyInPandas(mean_func, schema="user_id long, value double").show()
```

Utilize the mapInPandas API:

C)

```
df.mapInPandas(mean_func, schema="user_id long, value double").show()
```

Use a regular Spark UDF (User-Defined Function):

```
from pyspark.sql.functions import mean
df.groupBy('user_id').agg(mean('value')).show()
```

Create a Pandas UDF (User-Defined Function):

```
from pyspark.sql.functions import pandas_udf

@pandas_udf("double")
def mean_func(value: pd.Series) -> float:
    return value.mean()

df.groupby("user_id").agg(mean_func(df['value'])) .show()
```

A. Use the applyInPandas API:

```
df.groupby("user_id").applyInPandas(mean_func, schema="user_id long, value double").show()
```

B. Use the mapInPandas API:

```
df.mapInPandas(mean_func, schema="user_id long, value double").show()
```

C. Use a regular Spark UDF:

```
from pyspark.sql.functions import mean
df.groupBy("user_id").agg(mean("value")).show()
```

D. Use a Pandas UDF:

```
@pandas_udf("double")
def mean_func(value: pd.Series) -> float:
    return value.mean()
df.groupby("user_id").agg(mean_func(df["value"])).show()
```

Answer: A

Explanation:

The correct approach to perform a parallelized groupBy operation across Spark worker nodes using Pandas API is via applyInPandas. This function enables grouped map operations using Pandas logic in a distributed Spark environment. It applies a user-defined function to each group of data represented as a Pandas DataFrame.

As per the Databricks documentation:

"applyInPandas() allows for vectorized operations on grouped data in Spark. It applies a user-defined function to each group of a DataFrame and outputs a new DataFrame. This is the recommended approach for using Pandas logic across grouped data with parallel execution." Option A is correct and achieves this parallel execution.

Option B (mapInPandas) applies to the entire DataFrame, not grouped operations.

Option C uses built-in aggregation functions, which are efficient but not customizable with Pandas logic.

Option D creates a scalar Pandas UDF which does not perform a group-wise transformation. Therefore, to run a groupBy with parallel Pandas logic on Spark workers, Option A using applyInPandas is the only correct answer.

QUESTION NO: 90

A developer wants to test Spark Connect with an existing Spark application. What are the two alternative ways the developer can start a local Spark Connect server without changing their existing application code? (Choose 2 answers)

- A. Execute their pyspark shell with the option `--remote "https://localhost"`
- B. Execute their pyspark shell with the option `--remote "sc://localhost"`
- C. Set the environment variable `SPARK_REMOTE="sc://localhost"` before starting the pyspark shell
- D. Add `.remote("sc://localhost")` to their `SparkSession.builder` calls in their Spark code
- E. Ensure the Spark property `spark.connect.grpc.binding.port` is set to 15002 in the application code

Answer: B,C

Explanation:

Spark Connect enables decoupling of the client and Spark driver processes, allowing remote access. Spark supports configuring the remote Spark Connect server in multiple ways:

From Databricks and Spark documentation:

Option B (`--remote "sc://localhost"`) is a valid command-line argument for the pyspark shell to connect using Spark Connect.

Option C (setting `SPARK_REMOTE` environment variable) is also a supported method to configure the remote endpoint.

Option A is incorrect because Spark Connect uses the `sc://` protocol, not `https://`.

Option D requires modifying the code, which the question explicitly avoids.

Option E configures the port on the server side but doesn't start a client connection.

Final Answers: B and C

QUESTION NO: 91

6 of 55.

Which components of Apache Spark's Architecture are responsible for carrying out tasks when assigned to them?

- A. Driver Nodes
- B. Executors
- C. CPU Cores
- D. Worker Nodes

Answer: B

Explanation:

In Spark's distributed architecture:

The Driver Node coordinates the execution of a Spark application. It converts the logical plan into a physical plan of stages and tasks.

The Executors, running on Worker Nodes, are responsible for executing tasks assigned by the driver and storing data (in memory or disk) during execution.

Key point:

Executors are the active agents that perform the actual computations on data partitions. Each executor runs multiple tasks in parallel using available CPU cores.

Why the other options are incorrect:

A (Driver Nodes): The driver schedules tasks; it doesn't execute them.

C (CPU Cores): CPU cores execute within executors, but they are hardware, not Spark architectural components.

D (Worker Nodes): Worker nodes host executors but do not directly execute tasks; executors do.

Reference (Databricks Apache Spark 3.5 - Python / Study Guide):

Spark Architecture Components - Driver, Executors, Cluster Manager, Worker Nodes.

Databricks Exam Guide (June 2025): Section "Apache Spark Architecture and Components" - describes the roles of driver and executor nodes in distributed processing.

QUESTION NO: 92

How can a Spark developer ensure optimal resource utilization when running Spark jobs in Local Mode for testing?

Options:

- A. Configure the application to run in cluster mode instead of local mode.
- B. Increase the number of local threads based on the number of CPU cores.
- C. Use the `spark.dynamicAllocation.enabled` property to scale resources dynamically.
- D. Set the `spark.executor.memory` property to a large value.

Answer: B

Explanation:

When running in local mode (e.g., `local[4]`), the number inside the brackets defines how many threads Spark will use.

Using `local[*]` ensures Spark uses all available CPU cores for parallelism.

Example:

```
spark-submit --master local[*]
```

Dynamic allocation and executor memory apply to cluster-based deployments, not local mode.

QUESTION NO: 93

1 of 55. A data scientist wants to ingest a directory full of plain text files so that each record in the output DataFrame contains the entire contents of a single file and the full path of the file the text was read from.

The first attempt does read the text files, but each record contains a single line. This code is shown below:

```
txt_path = "/datasets/raw_txt/"
```

```
df = spark.read.text(txt_path) # one row per line by default
```

```
df = df.withColumn("file_path", input_file_name()) # add full path
```

Which code change can be implemented in a DataFrame that meets the data scientist's requirements?

- A. Add the option `wholesetext` to the `text()` function.
- B. Add the option `lineSep` to the `text()` function.
- C. Add the option `wholesetext=False` to the `text()` function.
- D. Add the option `lineSep=","` to the `text()` function.

Answer: A

Explanation:

By default, the `spark.read.text()` method reads a text file one line per record. This means that

each line in a text file becomes one row in the resulting DataFrame.

To read each file as a single record, Apache Spark provides the option `wholetext`, which, when set to `True`, causes Spark to treat the entire file contents as one single string per row.

Correct usage:

```
df = spark.read.option("wholetext", True).text(txt_path)
```

This way, each record in the DataFrame will contain the full content of one file instead of one line per record.

To also include the file path, the function `input_file_name()` can be used to create an additional column that stores the complete path of the file being read:

```
from pyspark.sql.functions import input_file_name
df = spark.read.option("wholetext", True).text(txt_path) \
    .withColumn("file_path", input_file_name())
```

This approach satisfies both requirements from the question:

Each record holds the entire contents of a file.

Each record also contains the file path from which the text was read.

Why the other options are incorrect:

B or D (`lineSep`) - The `lineSep` option only defines the delimiter between lines. It does not combine the entire file content into a single record.

C (`wholetext=False`) - This is the default behavior, which still reads one record per line rather than per file.

Reference (Databricks Apache Spark 3.5 - Python / Study Guide):

PySpark API Reference: `DataFrameReader.text` - describes the `wholetext` option.

PySpark Functions: `input_file_name()` - adds a column with the source file path.

Databricks Certified Associate Developer for Apache Spark Exam Guide (June 2025):

Section "Using Spark DataFrame APIs" - covers reading files and handling DataFrames.

QUESTION NO: 94

Given a CSV file with the content:

```
bambi,hello
alladin,20
```

Given the code fragment:

```
from pyspark.sql.types import *
schema = StructType([StructField("name", StringType()), StructField("age", IntegerType())])
spark.read.schema(schema).csv(path).collect()
```

And the following code:

```
from pyspark.sql.types import *
schema = StructType([
    StructField("name", StringType()),
    StructField("age", IntegerType())
])
spark.read.schema(schema).csv(path).collect()
```

What is the resulting output?

- A. [Row(name='bambi'), Row(name='alladin', age=20)]
- B. [Row(name='alladin', age=20)]

C. [Row(name='bambi', age=None), Row(name='alladin', age=20)]

D. The code throws an error due to a schema mismatch.

Answer: C

Explanation:

In Spark, when a CSV row does not match the provided schema, Spark does not raise an error by default. Instead, it returns null for fields that cannot be parsed correctly.

In the first row, "hello" cannot be cast to Integer for the age field → Spark sets age=None In the second row, "20" is a valid integer → age=20 So the output will be:

[Row(name='bambi', age=None), Row(name='alladin', age=20)]

Final answer: C

QUESTION NO: 95

A data engineer wants to create an external table from a JSON file located at /data/input.json with the following requirements:

Create an external table named users

Automatically infer schema

Merge records with differing schemas

Which code snippet should the engineer use?

Options:

A. CREATE TABLE users USING json OPTIONS (path '/data/input.json')

B. CREATE EXTERNAL TABLE users USING json OPTIONS (path '/data/input.json')

C. CREATE EXTERNAL TABLE users USING json OPTIONS (path '/data/input.json', mergeSchema 'true')

D. CREATE EXTERNAL TABLE users USING json OPTIONS (path '/data/input.json', schemaMerge 'true')

Answer: C

Explanation:

To create an external table and enable schema merging, the correct syntax is:

CREATE EXTERNAL TABLE users

USING json

OPTIONS (

path '/data/input.json',

mergeSchema 'true'

)

mergeSchema is the correct option key (not schemaMerge)

EXTERNAL allows Spark to query files without managing their lifecycle

QUESTION NO: 96

A data scientist wants each record in the DataFrame to contain:

The first attempt at the code does read the text files but each record contains a single line.

This code is shown below:

```
raw_txt_path = '/datasets/raw_txt/*'

corpus = spark.read.text(raw_txt_path) \
    .select('*', '_metadata.file_path')
```

The entire contents of a file

The full file path

The issue: reading line-by-line rather than full text per file.

Code:

```
corpus = spark.read.text("/datasets/raw_txt/*") \
    .select('*', '_metadata.file_path')
```

Which change will ensure one record per file?

Options:

- A. Add the option `wholertext=True` to the `text()` function
- B. Add the option `lineSep='\n'` to the `text()` function
- C. Add the option `wholertext=False` to the `text()` function
- D. Add the option `lineSep=", "` to the `text()` function

Answer: A

Explanation:

To read each file as a single record, use:

```
spark.read.text(path, wholertext=True)
```

This ensures that Spark reads the entire file contents into one row.

QUESTION NO: 97

A data scientist is working on a large dataset in Apache Spark using PySpark. The data scientist has a DataFrame `df` with columns `user_id`, `product_id`, and `purchase_amount` and needs to perform some operations on this data efficiently.

Which sequence of operations results in transformations that require a shuffle followed by transformations that do not?

- A. `df.filter(df.purchase_amount > 100).groupBy("user_id").sum("purchase_amount")`
- B. `df.withColumn("discount", df.purchase_amount * 0.1).select("discount")`
- C. `df.withColumn("purchase_date", current_date()).where("total_purchase > 50")`
- D. `df.groupBy("user_id").agg(sum("purchase_amount").alias("total_purchase")).repartition(10)`

Answer: D

Explanation:

Shuffling occurs in operations like `groupBy`, `reduceByKey`, or `join`-which cause data to be moved across partitions. The `repartition()` operation can also cause a shuffle, but in this context, it follows an aggregation.

In Option D, the `groupBy` followed by `agg` results in a shuffle due to grouping across nodes. The `repartition(10)` is a partitioning transformation but does not involve a new shuffle since

the data is already grouped.

This sequence - shuffle (groupBy) followed by non-shuffling (repartition) - is correct.

Option A does the opposite: the filter does not cause a shuffle, but groupBy does - this makes it the wrong order.

QUESTION NO: 98

25 of 55.

A Data Analyst is working on employees_df and needs to add a new column where a 10% tax is calculated on the salary.

Additionally, the DataFrame contains the column age, which is not needed.

Which code fragment adds the tax column and removes the age column?

A.

```
employees_df = employees_df.withColumn("tax", col("salary") * 0.1).drop("age")
```

B.

```
employees_df = employees_df.withColumn("tax", lit(0.1)).drop("age")
```

C.

```
employees_df = employees_df.dropField("age").withColumn("tax", col("salary") * 0.1)
```

D.

```
employees_df = employees_df.withColumn("tax", col("salary") + 0.1).drop("age")
```

Answer: A

Explanation:

To create a new calculated column in Spark, use the .withColumn() method.

To remove an unwanted column, use the .drop() method.

Correct syntax:

```
from pyspark.sql.functions import col
```

```
employees_df = employees_df.withColumn("tax", col("salary") * 0.1).drop("age")
```

.withColumn("tax", col("salary") * 0.1) → adds a new column where tax = 10% of salary.

.drop("age") → removes the age column from the DataFrame.

Why the other options are incorrect:

B: lit(0.1) creates a constant value, not a calculated tax.

C: .dropField() is not a DataFrame API method (used only in struct field manipulations).

D: Adds 0.1 to salary instead of calculating 10%.

Reference:

PySpark DataFrame API - withColumn(), drop(), and col().

Databricks Exam Guide (June 2025): Section "Developing Apache Spark

DataFrame/DataSet API Applications" - manipulating, renaming, and dropping columns.

QUESTION NO: 99

You have:

DataFrame A: 128 GB of transactions

DataFrame B: 1 GB user lookup table

Which strategy is correct for broadcasting?

A. DataFrame B should be broadcasted because it is smaller and will eliminate the need for shuffling itself

B. DataFrame B should be broadcasted because it is smaller and will eliminate the need for

shuffling DataFrame A

C. DataFrame A should be broadcasted because it is larger and will eliminate the need for shuffling DataFrame B

D. DataFrame A should be broadcasted because it is smaller and will eliminate the need for shuffling itself

Answer: B

Explanation:

Broadcast joins work by sending the smaller DataFrame to all executors, eliminating the shuffle of the larger DataFrame.

From Spark documentation:

"Broadcast joins are efficient when one DataFrame is small enough to fit in memory. Spark avoids shuffling the larger table." DataFrame B (1 GB) fits within the default threshold and should be broadcasted.

It eliminates the need to shuffle the large DataFrame A.

Final answer: B

QUESTION NO: 100

46 of 55.

A data engineer is implementing a streaming pipeline with watermarking to handle late-arriving records.

The engineer has written the following code:

```
inputStream \
.withWatermark("event_time", "10 minutes") \
.groupBy(window("event_time", "15 minutes"))
```

What happens to data that arrives after the watermark threshold?

A. Any data arriving more than 10 minutes after the watermark threshold will be ignored and not included in the aggregation.

B. Records that arrive later than the watermark threshold (10 minutes) will automatically be included in the aggregation if they fall within the 15-minute window.

C. Data arriving more than 10 minutes after the latest watermark will still be included in the aggregation but will be placed into the next window.

D. The watermark ensures that late data arriving within 10 minutes of the latest event time will be processed and included in the windowed aggregation.

Answer: A

Explanation:

Watermarking in Structured Streaming defines how late a record can arrive based on event time before Spark discards it.

Behavior:

```
.withWatermark("event_time", "10 minutes")
```

This means Spark will keep state for 10 minutes beyond the maximum event time seen so far.

Any data arriving later than 10 minutes after the current watermark is ignored - it will not be included in the aggregation or output.

Why the other options are incorrect:

B: Late data beyond the watermark threshold is not included.

C: Late data is not moved to a new window; it's simply dropped.

D: True for late data within the watermark threshold, not after it.

Reference:

Spark Structured Streaming Guide - withWatermark() behavior and late data handling.

Databricks Exam Guide (June 2025): Section "Structured Streaming" - watermarking and state cleanup behavior.

QUESTION NO: 101

43 of 55.

An organization has been running a Spark application in production and is considering disabling the Spark History Server to reduce resource usage.

What will be the impact of disabling the Spark History Server in production?

- A. Prevention of driver log accumulation during long-running jobs
- B. Improved job execution speed due to reduced logging overhead
- C. Loss of access to past job logs and reduced debugging capability for completed jobs
- D. Enhanced executor performance due to reduced log size

Answer: C

Explanation:

The Spark History Server provides a web UI for viewing past completed applications, including event logs, stages, and performance metrics.

If disabled:

Spark jobs still run normally,

But users lose the ability to review historical job metrics, DAGs, or logs after completion.

Thus, debugging, performance analysis, and audit capabilities are lost.

Why the other options are incorrect:

A: Disabling History Server doesn't manage logs.

B/D: Minimal overhead; disabling doesn't improve runtime speed or executor performance.

Reference:

Databricks Exam Guide (June 2025): Section "Apache Spark Architecture and Components" - Spark UI, History Server, and event logging.

Spark Administration Docs - History Server functionality and configuration.

QUESTION NO: 102

35 of 55.

A data engineer is building a Structured Streaming pipeline and wants it to recover from failures or intentional shutdowns by continuing where it left off.

How can this be achieved?

- A. By configuring the option recoveryLocation during SparkSession initialization.
- B. By configuring the option checkpointLocation during readStream.
- C. By configuring the option checkpointLocation during writeStream.
- D. By configuring the option recoveryLocation during writeStream.

Answer: C

Explanation:

In Structured Streaming, checkpoints store state information (offsets, progress, and metadata) needed to resume a stream after a failure or restart.

Correct usage:

Set the checkpointLocation option when writing the streaming output:

```
streaming_df.writeStream \
    .format("delta") \
    .option("checkpointLocation", "/path/to/checkpoint/dir") \
    .start("/path/to/output")
```

Spark uses this checkpoint directory to recover progress automatically and maintain exactly-once semantics.

Why the other options are incorrect:

A/D: recoveryLocation is not a valid Spark configuration option.

B: Checkpointing must be configured in writeStream, not during readStream.

Reference:

PySpark Structured Streaming Guide - Checkpointing and recovery.

Databricks Exam Guide (June 2025): Section "Structured Streaming" - explains checkpointing and fault-tolerant streaming recovery.

QUESTION NO: 103

A data scientist at a financial services company is working with a Spark DataFrame containing transaction records. The DataFrame has millions of rows and includes columns for transaction_id, account_number, transaction_amount, and timestamp. Due to an issue with the source system, some transactions were accidentally recorded multiple times with identical information across all fields. The data scientist needs to remove rows with duplicates across all fields to ensure accurate financial reporting.

Which approach should the data scientist use to deduplicate the orders using PySpark?

- A. `df = df.dropDuplicates()`
- B. `df = df.groupBy("transaction_id").agg(F.first("account_number"), F.first("transaction_amount"), F.first("timestamp"))`
- C. `df = df.filter(F.col("transaction_id").isNotNull())`
- D. `df = df.dropDuplicates(["transaction_amount"])`

Answer: A

Explanation:

`dropDuplicates()` with no column list removes duplicates based on all columns.

It's the most efficient and semantically correct way to deduplicate records that are completely identical across all fields.

From the PySpark documentation:

`dropDuplicates()`: Return a new DataFrame with duplicate rows removed, considering all columns if none are specified.

- Source: PySpark DataFrame.dropDuplicates() API

QUESTION NO: 104

Which Spark configuration controls the number of tasks that can run in parallel on the executor?

Options:

- A. `spark.executor.cores`
- B. `spark.task.maxFailures`

- C. spark.driver.cores
- D. spark.executor.memory

Answer: A

Explanation:

spark.executor.cores determines how many concurrent tasks an executor can run.

For example, if set to 4, each executor can run up to 4 tasks in parallel.

Other settings:

spark.task.maxFailures controls task retry logic.

spark.driver.cores is for the driver, not executors.

spark.executor.memory sets memory limits, not task concurrency.

QUESTION NO: 105

Given a DataFrame df that has 10 partitions, after running the code:

```
result = df.coalesce(20)
```

How many partitions will the result DataFrame have?

- A. 10
- B. Same number as the cluster executors
- C. 1
- D. 20

Answer: A

Explanation:

The .coalesce(numPartitions) function is used to reduce the number of partitions in a DataFrame. It does not increase the number of partitions. If the specified number of partitions is greater than the current number, it will not have any effect.

From the official Spark documentation:

"coalesce() results in a narrow dependency, e.g. if you go from 1000 partitions to 100 partitions, there will not be a shuffle, instead each of the 100 new partitions will claim one or more of the current partitions." However, if you try to increase partitions using coalesce (e.g., from 10 to 20), the number of partitions remains unchanged.

Hence, df.coalesce(20) will still return a DataFrame with 10 partitions.

QUESTION NO: 106

What is the relationship between jobs, stages, and tasks during execution in Apache Spark?

Options:

- A. A job contains multiple stages, and each stage contains multiple tasks.
- B. A job contains multiple tasks, and each task contains multiple stages.
- C. A stage contains multiple jobs, and each job contains multiple tasks.
- D. A stage contains multiple tasks, and each task contains multiple jobs.

Answer: A

Explanation:

A Spark job is triggered by an action (e.g., count, show).

The job is broken into stages, typically one per shuffle boundary.

Each stage is divided into multiple tasks, which are distributed across worker nodes.

QUESTION NO: 107

Which feature of Spark Connect is considered when designing an application to enable remote interaction with the Spark cluster?

- A.** It provides a way to run Spark applications remotely in any programming language
- B.** It can be used to interact with any remote cluster using the REST API
- C.** It allows for remote execution of Spark jobs
- D.** It is primarily used for data ingestion into Spark from external sources

Answer: C

Explanation:

Spark Connect introduces a decoupled client-server architecture. Its key feature is enabling Spark job submission and execution from remote clients - in Python, Java, etc.

From Databricks documentation:

"Spark Connect allows remote clients to connect to a Spark cluster and execute Spark jobs without being co-located with the Spark driver." A is close, but "any language" is overstated (currently supports Python, Java, etc., not literally all).

B refers to REST, which is not Spark Connect's mechanism.

D is incorrect; Spark Connect isn't focused on ingestion.

Final answer: C

QUESTION NO: 108

A data scientist is working with a Spark DataFrame called `customerDF` that contains customer information. The DataFrame has a column named `email` with customer email addresses. The data scientist needs to split this column into username and domain parts. Which code snippet splits the email column into username and domain columns?

A.

```
customerDF.select(
  col("email").substr(0, 5).alias("username"),
  col("email").substr(-5).alias("domain")
)
```

B.

```
customerDF.withColumn("username", split(col("email"), "@").getItem(0)) \
  .withColumn("domain", split(col("email"), "@").getItem(1))
```

C.

```
customerDF.withColumn("username", substring_index(col("email"), "@", 1)) \
  .withColumn("domain", substring_index(col("email"), "@", -1))
```

D.

```
customerDF.select(
  regexp_replace(col("email"), "@", "").alias("username"),
  regexp_replace(col("email"), "@", "").alias("domain")
)
```

Answer: B

Explanation:

Option B is the correct and idiomatic approach in PySpark to split a string column (like email) based on a delimiter such as "@".

The `split(col("email"), "@")` function returns an array with two elements: username and domain.

`getItem(0)` retrieves the first part (username).

`getItem(1)` retrieves the second part (domain).

`withColumn()` is used to create new columns from the extracted values.

Example from official Databricks Spark documentation on splitting columns:

```
from pyspark.sql.functions import split, col
df.withColumn("username", split(col("email"), "@").getItem(0)) \
  .withColumn("domain", split(col("email"), "@").getItem(1))
```

Why other options are incorrect:

A uses fixed substring indices (`substr(0, 5)`), which won't correctly extract usernames and domains of varying lengths.

C uses `substring_index`, which is available but less idiomatic for splitting emails and is slightly less readable.

D removes "@" from the email entirely, losing the separation between username and domain, and ends up duplicating values in both fields.

Therefore, Option B is the most accurate and reliable solution according to Apache Spark 3.5 best practices.

QUESTION NO: 109

A data engineer needs to persist a file-based data source to a specific location. However, by default, Spark writes to the warehouse directory (e.g., `/user/hive/warehouse`). To override this, the engineer must explicitly define the file path.

Which line of code ensures the data is saved to a specific location?

Options:

A. `users.write(path="/some/path").saveAsTable("default_table")`

B. `users.write.saveAsTable("default_table").option("path", "/some/path")`

C. `users.write.option("path", "/some/path").saveAsTable("default_table")`

D. `users.write.saveAsTable("default_table", path="/some/path")`

Answer: C

Explanation:

To persist a table and specify the save path, use:

```
users.write.option("path", "/some/path").saveAsTable("default_table")
```

The `.option("path", ...)` must be applied before calling `saveAsTable`.

Option A uses invalid syntax (`write(path=...)`).

Option B applies `.option()` after `.saveAsTable()`-which is too late.

Option D uses incorrect syntax (no path parameter in `saveAsTable`).

QUESTION NO: 110

22 of 55.

A Spark application needs to read multiple Parquet files from a directory where the files have differing but compatible schemas.

The data engineer wants to create a `DataFrame` that includes all columns from all files.

Which code should the data engineer use to read the Parquet files and include all columns using Apache Spark?

- A. `spark.read.parquet("/data/parquet/")`
- B. `spark.read.option("mergeSchema", True).parquet("/data/parquet/")`
- C. `spark.read.format("parquet").option("inferSchema", "true").load("/data/parquet/")`
- D. `spark.read.parquet("/data/parquet/").option("mergeAllCols", True)`

Answer: B

Explanation:

When reading Parquet files, Spark infers a unified schema automatically only if all files share identical structures.

If files have different but compatible schemas, you must enable schema merging by setting the option `mergeSchema=True`.

Correct syntax:

```
df = spark.read.option("mergeSchema", True).parquet("/data/parquet/")
```

This option ensures Spark merges all discovered fields across Parquet files into one unified DataFrame schema.

Why the other options are incorrect:

A: Loads files but ignores extra columns - uses only the first file's schema.

C: `inferSchema` applies to CSV/JSON, not Parquet.

D: `mergeAllCols` is not a valid Spark option.

Reference:

Spark SQL Data Sources - Parquet options (`mergeSchema`, `path`).

Databricks Exam Guide (June 2025): Section "Using Spark DataFrame APIs" - reading/writing DataFrames with schema evolution and merging.

QUESTION NO: 111

38 of 55.

A data engineer is working with Spark SQL and has a large JSON file stored at `/data/input.json`.

The file contains records with varying schemas, and the engineer wants to create an external table in Spark SQL that:

Reads directly from `/data/input.json`.

Infers the schema automatically.

Merges differing schemas.

Which code snippet should the engineer use?

A.

```
CREATE EXTERNAL TABLE users
USING json
OPTIONS (path '/data/input.json', mergeSchema 'true');
```

B.

```
CREATE TABLE users
USING json
OPTIONS (path '/data/input.json');
```

C.

```
CREATE EXTERNAL TABLE users
USING json
OPTIONS (path '/data/input.json', inferSchema 'true');
```

D.

```
CREATE EXTERNAL TABLE users
USING json
OPTIONS (path '/data/input.json', mergeAll 'true');
```

Answer: A

Explanation:

To handle JSON files with evolving or differing schemas, Spark SQL supports the option `mergeSchema 'true'`, which merges all fields across files into a unified schema.

Correct syntax:

```
CREATE EXTERNAL TABLE users
USING json
OPTIONS (path '/data/input.json', mergeSchema 'true');
```

This creates an external table directly on the JSON data, inferring schema automatically and merging variations.

Why the other options are incorrect:

B: Missing schema merge configuration - fails with inconsistent files.

C: `inferSchema` applies to CSV/other file types, not JSON.

D: `mergeAll` is not a valid Spark SQL option.

Reference:

Spark SQL Data Sources - JSON file options (`mergeSchema`, `path`).

Databricks Exam Guide (June 2025): Section "Using Spark SQL" - creating external tables and schema inference for JSON data.

QUESTION NO: 112

A data engineer wants to write a Spark job that creates a new managed table. If the table already exists, the job should fail and not modify anything.

Which save mode and method should be used?

A. saveAsTable with mode ErrorIfExists

B. saveAsTable with mode Overwrite

C. save with mode Ignore

D. save with mode ErrorIfExists

Answer: A

Explanation:

The method `saveAsTable()` creates a new table and optionally fails if the table exists.

From Spark documentation:

"The mode 'ErrorIfExists' (default) will throw an error if the table already exists." Thus:

Option A is correct.

Option B (Overwrite) would overwrite existing data - not acceptable here.

Option C and D use `save()`, which doesn't create a managed table with metadata in the metastore.

Final answer: A

QUESTION NO: 113

A Data Analyst needs to retrieve employees with 5 or more years of tenure.

Which code snippet filters and shows the list?

- A. `employees_df.filter(employees_df.tenure >= 5).show()`
- B. `employees_df.where(employees_df.tenure >= 5)`
- C. `filter(employees_df.tenure >= 5)`
- D. `employees_df.filter(employees_df.tenure >= 5).collect()`

Answer: A

Explanation:

To filter rows based on a condition and display them in Spark, use `filter(...).show()`:

`employees_df.filter(employees_df.tenure >= 5).show()`

Option A is correct and shows the results.

Option B filters but doesn't display them.

Option C uses Python's built-in filter, not Spark.

Option D collects the results to the driver, which is unnecessary if `.show()` is sufficient.

Final answer: A

QUESTION NO: 114

42 of 55.

A developer needs to write the output of a complex chain of Spark transformations to a Parquet table called `events.liveLatest`.

Consumers of this table query it frequently with filters on both year and month of the `event_ts` column (a timestamp).

The current code:

```
from pyspark.sql import functions as F
final = df.withColumn("event_year", F.year("event_ts")) \
.withColumn("event_month", F.month("event_ts")) \
.bucketBy(42, ["event_year", "event_month"]) \
.saveAsTable("events.liveLatest")
```

However, consumers report poor query performance.

Which change will enable efficient querying by year and month?

- A. Replace `.bucketBy()` with `.partitionBy("event_year", "event_month")`
- B. Change the bucket count (42) to a lower number
- C. Add `.sortBy()` after `.bucketBy()`
- D. Replace `.bucketBy()` with `.partitionBy("event_year")` only

Answer: A

Explanation:

When queries frequently filter on certain columns, partitioning by those columns ensures partition pruning, allowing Spark to scan only relevant directories instead of the entire dataset.

Correct code:

`final.write.partitionBy("event_year", "event_month").parquet("events.liveLatest")` This improves read performance dramatically for filters like:

`SELECT * FROM events.liveLatest WHERE event_year = 2024 AND event_month = 5;`

`bucketBy()` helps in clustering and joins, not in partition pruning for file-based tables.

Why the other options are incorrect:

B: Bucket count changes parallelism, not query pruning.

C: sortBy organizes data within files, not across partitions.

D: Partitioning by only one column limits pruning benefits.

Reference:

Spark SQL DataFrameWriter - partitionBy() for partitioned tables.

Databricks Exam Guide (June 2025): Section "Using Spark SQL" - partitioning vs. bucketing and query optimization.

QUESTION NO: 115

28 of 55.

A data analyst builds a Spark application to analyze finance data and performs the following operations:

filter, select, groupBy, and coalesce.

Which operation results in a shuffle?

A. filter

B. select

C. groupBy

D. coalesce

Answer: C

Explanation:

In Spark, a shuffle occurs when data needs to be redistributed across partitions or nodes, typically due to operations that require grouping, joining, or sorting.

groupBy triggers a shuffle because it aggregates data based on key values, requiring data from multiple partitions to be moved across the cluster.

Operations like filter and select are narrow transformations (no shuffle).

coalesce can reduce the number of partitions without a full shuffle (unlike repartition).

Why the other options are incorrect:

A (filter) - Narrow transformation, no shuffle.

B (select) - Simple projection, no data movement.

D (coalesce) - Reduces partitions locally without shuffle.

Reference:

Spark Architecture - Transformations and Actions; narrow vs. wide transformations.

Databricks Exam Guide (June 2025): Section "Apache Spark Architecture and Components" - explains shuffle operations and execution hierarchy.

QUESTION NO: 116

15 of 55.

A data engineer is working on a Streaming DataFrame (streaming_df) with the following streaming data:

id

name

count

timestamp

1

Delhi

20

2024-09-19T10:11

1

Delhi

50

2024-09-19T10:12

2

London

50

2024-09-19T10:15

3

Paris

30

2024-09-19T10:18

3

Paris

20

2024-09-19T10:20

4

Washington

10

2024-09-19T10:22

Which operation is supported with streaming_df?

- A. streaming_df.count()
- B. streaming_df.filter("count < 30")
- C. streaming_df.select(countDistinct("name"))
- D. streaming_df.show()

Answer: B

Explanation:

In Structured Streaming, only transformation operations are allowed on streaming DataFrames. These include select(), filter(), where(), groupBy(), withColumn(), etc.

Example of supported transformation:

```
filtered_df = streaming_df.filter("count < 30")
```

However, actions such as count(), show(), and collect() are not supported directly on streaming DataFrames because streaming queries are unbounded and never finish until stopped.

To perform aggregations, the query must be executed through writeStream and an output sink.

Why the other options are incorrect:

A: count() is an action, not allowed directly on streaming DataFrames.

C: countDistinct() is a stateful aggregation, not supported outside of a proper streaming query.

D: show() is also an action, unsupported on streaming queries.

Reference:

PySpark Structured Streaming Programming Guide - supported transformations and actions.

Databricks Exam Guide (June 2025): Section "Structured Streaming" - performing operations

on streaming DataFrames and understanding supported transformations.

QUESTION NO: 117

A data engineer is building an Apache Spark™ Structured Streaming application to process a stream of JSON events in real time. The engineer wants the application to be fault-tolerant and resume processing from the last successfully processed record in case of a failure. To achieve this, the data engineer decides to implement checkpoints.

Which code snippet should the data engineer use?

A.

```
query = streaming_df.writeStream \
    .format("console") \
    .option("checkpoint", "/path/to/checkpoint") \
    .outputMode("append") \
    .start()
```

B.

```
query = streaming_df.writeStream \
    .format("console") \
    .outputMode("append") \
    .option("checkpointLocation", "/path/to/checkpoint") \
    .start()
```

C.

```
query = streaming_df.writeStream \
    .format("console") \
    .outputMode("complete") \
    .start()
```

D.

```
query = streaming_df.writeStream \
    .format("console") \
    .outputMode("append") \
    .start()
```

Answer: B

Explanation:

To enable fault tolerance and ensure that Spark can resume from the last committed offset after failure, you must configure a checkpoint location using the correct option key: "checkpointLocation".

From the official Spark Structured Streaming guide:

"To make a streaming query fault-tolerant and recoverable, a checkpoint directory must be specified using .option("checkpointLocation", "/path/to/dir")." Explanation of options:

Option A uses an invalid option name: "checkpoint" (should be "checkpointLocation") Option B is correct: it sets checkpointLocation properly Option C lacks checkpointing and won't resume after failure Option D also lacks checkpointing configuration

QUESTION NO: 118

45 of 55.

Which feature of Spark Connect should be considered when designing an application that

plans to enable remote interaction with a Spark cluster?

- A. It is primarily used for data ingestion into Spark from external sources.
- B. It provides a way to run Spark applications remotely in any programming language.
- C. It can be used to interact with any remote cluster using the REST API.
- D. It allows for remote execution of Spark jobs.

Answer: D

Explanation:

Spark Connect enables remote execution of Spark jobs by decoupling the client from the driver using the Spark Connect protocol (gRPC).

It allows users to run Spark code from different environments (like notebooks, IDEs, or remote clients) while executing jobs on the cluster.

Key Features:

Enables remote interaction between client and Spark driver.

Supports interactive development and lightweight client sessions.

Improves developer productivity without needing driver resources locally.

Why the other options are incorrect:

A: Spark Connect is not limited to ingestion tasks.

B: It allows multi-language clients (Python, Scala, etc.) but runs via Spark Connect API, not arbitrary remote code.

C: Uses gRPC protocol, not REST.

Reference:

Databricks Exam Guide (June 2025): Section "Using Spark Connect to Deploy Applications" - describes Spark Connect architecture and remote execution model.

Spark 3.5 Documentation - Spark Connect overview and client-server protocol.

QUESTION NO: 119

Given this view definition:

```
df.createOrReplaceTempView("users_vw")
```

Which approach can be used to query the users_vw view after the session is terminated?

Options:

- A. Query the users_vw using Spark
- B. Persist the users_vw data as a table
- C. Recreate the users_vw and query the data using Spark
- D. Save the users_vw definition and query using Spark

Answer: B

Explanation:

Temp views like createOrReplaceTempView are session-scoped.

They disappear once the Spark session ends.

To retain data across sessions, it must be persisted:

```
df.write.saveAsTable("users_vw")
```

Thus, the view needs to be persisted as a table to survive session termination.

QUESTION NO: 120

39 of 55.

A Spark developer is developing a Spark application to monitor task performance across a

cluster.

One requirement is to track the maximum processing time for tasks on each worker node and consolidate this information on the driver for further analysis.

Which technique should the developer use?

- A. Broadcast a variable to share the maximum time among workers.
- B. Configure the Spark UI to automatically collect maximum times.
- C. Use an RDD action like reduce() to compute the maximum time.
- D. Use an accumulator to record the maximum time on the driver.

Answer: C

Explanation:

RDD actions like reduce() aggregate values across all partitions and return the result to the driver.

To compute the maximum processing time, reduce() is ideal because it combines results from all tasks efficiently.

Example:

```
max_time = rdd_times.reduce(lambda x, y: max(x, y))
```

This aggregates maximum values from all executors into a single result on the driver.

Why the other options are incorrect:

- A: Broadcast variables distribute read-only data; they cannot aggregate results.
- B: Spark UI provides visualization, not programmatic collection.
- D: Accumulators support additive operations only (e.g., counters, sums), not non-associative ones like max.

Reference:

Spark RDD API - reduce() for aggregations.

Databricks Exam Guide (June 2025): Section "Apache Spark Architecture and Components" - actions, accumulators, and broadcast variables.

QUESTION NO: 121

40 of 55.

A developer wants to refactor older Spark code to take advantage of built-in functions introduced in Spark 3.5.

The original code:

```
from pyspark.sql import functions as F
```

```
min_price = 110.50
```

```
result_df = prices_df.filter(F.col("price") > min_price).agg(F.count("*"))
```

Which code block should the developer use to refactor the code?

- A. `result_df = prices_df.filter(F.col("price") > F.lit(min_price)).agg(F.count("*"))`
- B. `result_df = prices_df.where(F.lit("price") > min_price).groupBy().count()`
- C. `result_df = prices_df.withColumn("valid_price", when(col("price") > F.lit(min_price), True))`
- D. `result_df = prices_df.filter(F.lit(min_price) > F.col("price")).count()`

Answer: A

Explanation:

To compare a column value with a Python literal constant in a DataFrame expression, use F.lit() to convert it into a Spark literal.

Correct refactor:

```
from pyspark.sql import functions as F
```

```
min_price = 110.50
```

```
result_df = prices_df.filter(F.col("price") > F.lit(min_price)).agg(F.count("*"))
```

This avoids type mismatches and ensures Spark executes the filter expression on the cluster.

Why the other options are incorrect:

B: where() syntax is valid, but F.lit("price") is incorrect - wraps string literal, not a column.

C: withColumn adds a column, not needed for this aggregation.

D: Comparison logic reversed.

Reference:

PySpark SQL Functions - lit(), col(), and DataFrame filters.

Databricks Exam Guide (June 2025): Section "Developing Apache Spark DataFrame/DataSet API Applications" - filtering, literals, and aggregations.

QUESTION NO: 122

A Data Analyst is working on the DataFrame sensor_df, which contains two columns: Which code fragment returns a DataFrame that splits the record column into separate columns and has one array item per row?

A)

```
exploded_df = sensor_df.withColumn("record_exploded", explode("record"))

exploded_df = exploded_df.select( "record_datetime", "sensor_id", "status", "health" )
exploded_df = sensor_df.withColumn("record_exploded", col("record"))
```

B)

```
exploded_df = exploded_df.select( "record_datetime", "record_exploded.sensor_id",
"record_exploded.status", "record_exploded.health" )
exploded_df = sensor_df.withColumn("record_exploded", explode("record"))
```

C)

```
exploded_df = exploded_df.select( "record_datetime", "record_exploded.sensor_id",
"record_exploded.status", "record_exploded.health" )
exploded_df = sensor_df.withColumn("record_exploded", explode("record"))
```

D)

```
exploded_df = exploded_df.select( "record_datetime", "record_exploded" )
```

A.

```
exploded_df = sensor_df.withColumn("record_exploded", explode("record")) exploded_df =
exploded_df.select("record_datetime", "sensor_id", "status", "health")
```

B.

```
exploded_df = exploded_df.select(
"record_datetime",
"record_exploded.sensor_id",
"record_exploded.status",
"record_exploded.health"
)
```

```
exploded_df = sensor_df.withColumn("record_exploded", explode("record"))
```

C.

```
exploded_df = exploded_df.select(
```

```

"record_datetime",
"record_exploded.sensor_id",
"record_exploded.status",
"record_exploded.health"
)
exploded_df = sensor_df.withColumn("record_exploded", explode("record"))

```

D.

```
exploded_df = exploded_df.select("record_datetime", "record_exploded")
```

Answer: C

Explanation:

To flatten an array of structs into individual rows and access fields within each struct, you must:

Use `explode()` to expand the array so each struct becomes its own row.

Access the struct fields via dot notation (e.g., `record_exploded.sensor_id`).

Option C does exactly that:

First, `explode` the record array column into a new column `record_exploded`.

Then, access fields of the struct using the dot syntax in `select`.

This is standard practice in PySpark for nested data transformation.

Final answer: C

QUESTION NO: 123

Given:

python

CopyEdit

```
spark.sparkContext.setLogLevel("<LOG_LEVEL>")
```

Which set contains the suitable configuration settings for Spark driver LOG_LEVELs?

A. ALL, DEBUG, FAIL, INFO

B. ERROR, WARN, TRACE, OFF

C. WARN, NONE, ERROR, FATAL

D. FATAL, NONE, INFO, DEBUG

Answer: B

Explanation:

The `setLogLevel()` method of `SparkContext` sets the logging level on the driver, which controls the verbosity of logs emitted during job execution. Supported levels are inherited from `log4j` and include the following:

ALL

DEBUG

ERROR

FATAL

INFO

OFF

TRACE

WARN

According to official Spark and Databricks documentation:

"Valid log levels include: ALL, DEBUG, ERROR, FATAL, INFO, OFF, TRACE, and WARN."

Among the choices provided, only option B (ERROR, WARN, TRACE, OFF) includes four valid log levels and excludes invalid ones like "FAIL" or "NONE".

QUESTION NO: 124

The following code fragment results in an error:

```
@F.udf(T.IntegerType())
def simple_udf(t: str) -> str:
return answer * 3.14159
```

Which code fragment should be used instead?

A.

```
@F.udf(T.IntegerType())
def simple_udf(t: int) -> int:
return t * 3.14159
```

B.

```
@F.udf(T.DoubleType())
def simple_udf(t: float) -> float:
return t * 3.14159
```

C.

```
@F.udf(T.DoubleType())
def simple_udf(t: int) -> int:
return t * 3.14159
```

D.

```
@F.udf(T.IntegerType())
def simple_udf(t: float) -> float:
return t * 3.14159
```

Answer: B

Explanation:

The original code has several issues:

It references a variable `answer` that is undefined.

The function is annotated to return a `str`, but the logic attempts numeric multiplication.

The UDF return type is declared as `T.IntegerType()` but the function performs a floating-point operation, which is incompatible.

Option B correctly:

Uses `DoubleType` to reflect the fact that the multiplication involves a float (3.14159).

Declares the input as `float`, which aligns with the multiplication.

Returns a `float`, which matches both the logic and the schema type annotation.

This structure aligns with how PySpark expects User Defined Functions (UDFs) to be declared:

"To define a UDF you must specify a Python function and provide the return type using the relevant Spark SQL type (e.g., `DoubleType` for float results)." Example from official documentation:

```
from pyspark.sql.functions import udf
from pyspark.sql.types import DoubleType
@udf(returnType=DoubleType())
def multiply_by_pi(x: float) -> float:
```

return x * 3.14159

This makes Option B the syntactically and semantically correct choice.

QUESTION NO: 125

34 of 55.

A data engineer is investigating a Spark cluster that is experiencing underutilization during scheduled batch jobs.

After checking the Spark logs, they noticed that tasks are often getting killed due to timeout errors, and there are several warnings about insufficient resources in the logs.

Which action should the engineer take to resolve the underutilization issue?

- A.** Set the spark.network.timeout property to allow tasks more time to complete without being killed.
- B.** Increase the executor memory allocation in the Spark configuration.
- C.** Reduce the size of the data partitions to improve task scheduling.
- D.** Increase the number of executor instances to handle more concurrent tasks.

Answer: D

Explanation:

Underutilization with timeout warnings often indicates insufficient parallelism - meaning there aren't enough executors to process all tasks concurrently.

Solution:

Increase the number of executors to allow more parallel task execution and better resource utilization.

Example configuration:

```
--conf spark.executor.instances=8
```

This distributes the workload more effectively across cluster nodes and reduces idle time for pending tasks.

Why the other options are incorrect:

A: Extending timeouts hides the symptom, not the root cause (lack of executors).

B: More memory per executor won't fix scheduling bottlenecks.

C: Reducing partition size may increase overhead and does not fix resource imbalance.

Reference:

Databricks Exam Guide (June 2025): Section "Troubleshooting and Tuning Apache Spark DataFrame API Applications" - tuning executors and cluster utilization.

Spark Configuration - executor instances and resource scaling.

QUESTION NO: 126

19 of 55.

A Spark developer wants to improve the performance of an existing PySpark UDF that runs a hash function not available in the standard Spark functions library.

The existing UDF code is:

```
import hashlib
from pyspark.sql.types import StringType
def shake_256(raw):
    return hashlib.shake_256(raw.encode()).hexdigest(20)
shake_256_udf = udf(shake_256, StringType())
```


The developer replaces this UDF with a Pandas UDF for better performance:

```
@pandas_udf(StringType())
def shake_256(raw: str) -> str:
    return hashlib.shake_256(raw.encode()).hexdigest(20)
```

However, the developer receives this error:

TypeError: Unsupported signature: (raw: str) -> str

What should the signature of the shake_256() function be changed to in order to fix this error?

A.

```
def shake_256(raw: str) -> str:
```

B.

```
def shake_256(raw: [pd.Series]) -> pd.Series:
```

C.

```
def shake_256(raw: pd.Series) -> pd.Series:
```

D.

```
def shake_256(raw: [str]) -> [str]:
```

Answer: C

Explanation:

Pandas UDFs (vectorized UDFs) process entire Pandas Series objects, not scalar values. Each invocation operates on a column (Series) rather than a single value.

Correct syntax:

```
@pandas_udf(StringType())
def shake_256(raw: pd.Series) -> pd.Series:
    return raw.apply(lambda x: hashlib.shake_256(x.encode()).hexdigest(20))
```

This allows Spark to apply the function in a vectorized way, improving performance significantly over traditional Python UDFs.

Why the other options are incorrect:

A/D: These define scalar functions - not compatible with Pandas UDFs.

B: Uses an invalid type hint [pd.Series] (not a valid Python type annotation).

Reference:

PySpark Pandas API - @pandas_udf decorator and function signatures.

Databricks Exam Guide (June 2025): Section "Using Pandas API on Apache Spark" - creating and invoking Pandas UDFs.

QUESTION NO: 127

A Spark developer is building an app to monitor task performance. They need to track the maximum task processing time per worker node and consolidate it on the driver for analysis. Which technique should be used?

A. Use an RDD action like reduce() to compute the maximum time

B. Use an accumulator to record the maximum time on the driver

C. Broadcast a variable to share the maximum time among workers

D. Configure the Spark UI to automatically collect maximum times

Answer: A

Explanation:

The correct way to aggregate information (e.g., max value) from distributed workers back to the driver is using RDD actions such as `reduce()` or `aggregate()`.

From the documentation:

"To perform global aggregations on distributed data, actions like `reduce()` are commonly used to collect summaries such as min/max/avg." Accumulators (Option B) do not support max operations directly and are not intended for such analytics.

Broadcast (Option C) is used to send data to workers, not collect from them.

Spark UI (Option D) is a monitoring tool - not an analytics collection interface.

Final answer: A

QUESTION NO: 128

A data engineer is working on a real-time analytics pipeline using Apache Spark Structured Streaming. The engineer wants to process incoming data and ensure that triggers control when the query is executed. The system needs to process data in micro-batches with a fixed interval of 5 seconds.

Which code snippet the data engineer could use to fulfil this requirement?

A)

```
query = df.writeStream \
    .outputMode("append") \
    .trigger(continuous='5 seconds') \
    .start()
```

B)

```
query = df.writeStream \
    .outputMode("append") \
    .trigger() \
    .start()
```

C)

```
query = df.writeStream \
    .outputMode("append") \
    .trigger(processingTime='5 seconds') \
    .start()
```

D)

```
query = df.writeStream \
    .outputMode("append") \
    .trigger(processingTime=5000) \
    .start()
```

Options:

- A. Uses trigger(continuous='5 seconds') - continuous processing mode.
- B. Uses trigger() - default micro-batch trigger without interval.
- C. Uses trigger(processingTime='5 seconds') - correct micro-batch trigger with interval.
- D. Uses trigger(processingTime=5000) - invalid, as processingTime expects a string.

Answer: C

Explanation:

To define a micro-batch interval, the correct syntax is:

```
query = df.writeStream \
    .outputMode("append") \
    .trigger(processingTime='5 seconds') \
    .start()
```

This schedules the query to execute every 5 seconds.

Continuous mode (used in Option A) is experimental and has limited sink support.

Option D is incorrect because processingTime must be a string (not an integer).

Option B triggers as fast as possible without interval control.

QUESTION NO: 129

A data scientist is working on a project that requires processing large amounts of structured data, performing SQL queries, and applying machine learning algorithms. The data scientist is considering using Apache Spark for this task.

Which combination of Apache Spark modules should the data scientist use in this scenario?

Options:

- A. Spark DataFrames, Structured Streaming, and GraphX
- B. Spark SQL, Pandas API on Spark, and Structured Streaming
- C. Spark Streaming, GraphX, and Pandas API on Spark
- D. Spark DataFrames, Spark SQL, and MLlib

Answer: D

Explanation:

Comprehensive

To cover structured data processing, SQL querying, and machine learning in Apache Spark, the correct combination of components is:

Spark DataFrames: for structured data processing

Spark SQL: to execute SQL queries over structured data

MLlib: Spark's scalable machine learning library

This trio is designed for exactly this type of use case.

Why other options are incorrect:

- A: GraphX is for graph processing - not needed here.
 B: Pandas API on Spark is useful, but MLlib is essential for ML, which this option omits.
 C: Spark Streaming is legacy; GraphX is irrelevant here.

QUESTION NO: 130

44 of 55.

A data engineer is working on a real-time analytics pipeline using Spark Structured Streaming.

They want the system to process incoming data in micro-batches at a fixed interval of 5 seconds.

Which code snippet fulfills this requirement?

A.

```
query = df.writeStream \
  .outputMode("append") \
  .trigger(processingTime="5 seconds") \
  .start()
```

B.

```
query = df.writeStream \
  .outputMode("append") \
  .trigger(continuous="5 seconds") \
  .start()
```

C.

```
query = df.writeStream \
  .outputMode("append") \
  .trigger(once=True) \
  .start()
```

D.

```
query = df.writeStream \
  .outputMode("append") \
  .start()
```

Answer: A

Explanation:

To process data in fixed micro-batch intervals, use the `.trigger(processingTime="interval")` option in Structured Streaming.

Correct usage:

```
query = df.writeStream \
  .outputMode("append") \
  .trigger(processingTime="5 seconds") \
  .start()
```

This instructs Spark to process available data every 5 seconds.

Why the other options are incorrect:

B: continuous triggers are for continuous processing mode (different execution model).

C: `once=True` runs the stream a single time (batch mode).

D: Default trigger runs as fast as possible, not fixed intervals.

Reference:

PySpark Structured Streaming Guide - Trigger types: processingTime, once, continuous.
Databricks Exam Guide (June 2025): Section "Structured Streaming" - controlling streaming triggers and batch intervals.

QUESTION NO: 131

54 of 55.

What is the benefit of Adaptive Query Execution (AQE)?

- A.** It allows Spark to optimize the query plan before execution but does not adapt during runtime.
- B.** It automatically distributes tasks across nodes in the clusters and does not perform runtime adjustments to the query plan.
- C.** It optimizes query execution by parallelizing tasks and does not adjust strategies based on runtime metrics like data skew.
- D.** It enables the adjustment of the query plan during runtime, handling skewed data, optimizing join strategies, and improving overall query performance.

Answer: D

Explanation:

Adaptive Query Execution (AQE) is a Spark SQL feature introduced to dynamically optimize queries at runtime based on actual data statistics collected during execution.

Key benefits include:

Runtime plan adaptation: Spark adjusts the physical plan after some stages complete.

Skew handling: Automatically splits skewed partitions to balance work distribution.

Join strategy optimization: Dynamically switches between shuffle join and broadcast join depending on partition sizes.

Coalescing shuffle partitions: Reduces the number of small tasks for better performance.

Example configuration:

```
spark.conf.set("spark.sql.adaptive.enabled", True)
```

This enables AQE globally in Spark 3.5.

Why the other options are incorrect:

- A:** AQE adapts during runtime, not only before execution.
- B:** Task distribution is a base Spark feature, not specific to AQE.
- C:** AQE specifically addresses runtime skew and join adjustments.

Reference:

Spark SQL Adaptive Query Execution Guide - Runtime optimization, skew handling, and join strategy adjustment.

Databricks Exam Guide (June 2025): Section "Troubleshooting and Tuning Apache Spark DataFrame API Applications" - Adaptive Query Execution benefits and configuration.

QUESTION NO: 132

A data engineer is working with a large JSON dataset containing order information. The dataset is stored in a distributed file system and needs to be loaded into a Spark DataFrame for analysis. The data engineer wants to ensure that the schema is correctly defined and that the data is read efficiently.

Which approach should the data scientist use to efficiently load the JSON data into a Spark DataFrame with a predefined schema?

- A.** Use `spark.read.json()` to load the data, then use `DataFrame.printSchema()` to view the inferred schema, and finally use `DataFrame.cast()` to modify column types.
- B.** Use `spark.read.json()` with the `inferSchema` option set to `true`
- C.** Use `spark.read.format("json").load()` and then use `DataFrame.withColumn()` to cast each column to the desired data type.
- D.** Define a `StructType` schema and use `spark.read.schema(predefinedSchema).json()` to load the data.

Answer: D

Explanation:

The most efficient and correct approach is to define a schema using `StructType` and pass it to `spark.read.schema(...)`.

This avoids schema inference overhead and ensures proper data types are enforced during read.

Example:

```
from pyspark.sql.types import StructType, StructField, StringType, DoubleType
schema = StructType([ StructField("order_id", StringType(), True),
                    StructField("amount", DoubleType(), True),
                    ...
                    ])
df = spark.read.schema(schema).json("path/to/json")
```

...

])

df = spark.read.schema(schema).json("path/to/json")

- Source: Databricks Guide - Read JSON with predefined schema

QUESTION NO: 133

What is a feature of Spark Connect?

- A.** It supports `DataStreamReader`, `DataStreamWriter`, `StreamingQuery`, and `Streaming APIs`
- B.** Supports `DataFrame`, `Functions`, `Column`, `SparkContext` `PySpark APIs`
- C.** It supports only `PySpark` applications
- D.** It has built-in authentication

Answer: A

Explanation:

Spark Connect is a client-server architecture introduced in Apache Spark 3.4, designed to decouple the client from the Spark driver, enabling remote connectivity to Spark clusters.

According to the Spark 3.5.5 documentation:

"Majority of the Streaming API is supported, including `DataStreamReader`, `DataStreamWriter`, `StreamingQuery` and `StreamingQueryListener`." This indicates that Spark Connect supports key components of Structured Streaming, allowing for robust streaming data processing capabilities.

Regarding other options:

B . While Spark Connect supports `DataFrame`, `Functions`, and `Column APIs`, it does not support `SparkContext` and `RDD APIs`.

C . Spark Connect supports multiple languages, including `PySpark` and `Scala`, not just `PySpark`.

D . Spark Connect does not have built-in authentication but is designed to work seamlessly with existing authentication infrastructures.

QUESTION NO: 134

Which configuration can be enabled to optimize the conversion between Pandas and PySpark DataFrames using Apache Arrow?

- A. `spark.conf.set("spark.pandas.arrow.enabled", "true")`
- B. `spark.conf.set("spark.sql.execution.arrow.pyspark.enabled", "true")`
- C. `spark.conf.set("spark.sql.execution.arrow.enabled", "true")`
- D. `spark.conf.set("spark.sql.arrow.pandas.enabled", "true")`

Answer: B

Explanation:

Apache Arrow is used under the hood to optimize conversion between Pandas and PySpark DataFrames. The correct configuration setting is:

`spark.conf.set("spark.sql.execution.arrow.pyspark.enabled", "true")`

From the official documentation:

"This configuration must be enabled to allow for vectorized execution and efficient conversion between Pandas and PySpark using Arrow." Option B is correct.

Options A, C, and D are invalid config keys and not recognized by Spark.

Final answer: B

QUESTION NO: 135

A data engineer has been asked to produce a Parquet table which is overwritten every day with the latest data. The downstream consumer of this Parquet table has a hard requirement that the data in this table is produced with all records sorted by the `market_time` field.

Which line of Spark code will produce a Parquet table that meets these requirements?

A.

```
final_df \
.sort("market_time") \
.write \
.format("parquet") \
.mode("overwrite") \
.saveAsTable("output.market_events")
```

B.

```
final_df \
.orderBy("market_time") \
.write \
.format("parquet") \
.mode("overwrite") \
.saveAsTable("output.market_events")
```

C.

```
final_df \
.sort("market_time") \
.coalesce(1) \
.write \
.format("parquet") \
.mode("overwrite") \
.saveAsTable("output.market_events")
```

D.

```
final_df \
.sortWithinPartitions("market_time") \
.write \
.format("parquet") \
.mode("overwrite") \
.saveAsTable("output.market_events")
```

Answer: D

Explanation:

To ensure that data written out to disk is sorted, it is important to consider how Spark writes data when saving to Parquet tables. The methods `.sort()` or `.orderBy()` apply a global sort but do not guarantee that the sorting will persist in the final output files unless certain conditions are met (e.g. a single partition via `.coalesce(1)` - which is not scalable).

Instead, the proper method in distributed Spark processing to ensure rows are sorted within their respective partitions when written out is:

```
.sortWithinPartitions("column_name")
```

According to Apache Spark documentation:

"`sortWithinPartitions()` ensures each partition is sorted by the specified columns. This is useful for downstream systems that require sorted files." This method works efficiently in distributed settings, avoids the performance bottleneck of global sorting (as in `.orderBy()` or `.sort()`), and guarantees each output partition has sorted records - which meets the requirement of consistently sorted data.

Thus:

Option A and B do not guarantee the persisted file contents are sorted.

Option C introduces a bottleneck via `.coalesce(1)` (single partition).

Option D correctly applies sorting within partitions and is scalable.